

Verificação de Restrições Operacionais em Agentes de Visão Computacional via Autômatos de Monitoramento: Especificação em LTL e TLTL para Apontamento de Produção em Manufatura

Flávio Miozzi Batista

¹Programa de Pós-Graduação em Computação Aplicada (PPComp)
Instituto Federal do Espírito Santo (Ifes) — Campus Serra, ES, Brasil

fmiozzi@gmail.com

Abstract. *Industrial vision agents now bridge the shop floor and the enterprise resource planning (ERP) layer, taking operational decisions with accounting impact on stock balance and raw-material consumption. This paper specifies four operational constraints — three as Linear Temporal Logic (LTL) and Timed LTL (TLTL) formulae, one as a structural filter — that a vision agent must satisfy while validating production reporting during the loading window of one arm of a multi-arm rotomoulding machine. From these specifications we synthesise monitoring automata via the Spot library, compose them by synchronous product, and state a compositional correctness proposition under both the ω -regular and the three-valued LTL_3 semantics. Three architectural components are formally specified: the composed monitor; a mes-bridge pre-processor that materialises the observed multiset and emits match/divergence events; and the MES $\rightarrow$ ERP integration gate. The divergence signal div_i is a derived event, materialised by the mes-bridge from the multiset comparison and from structural cycle exceptions; and the violation sinks of the component monitors are promoted to synthetic diagnostic events — a monitor-output-as-event interface that keeps the monitor declarative while exposing the architectural layer at which each failure occurs. The composed monitor decides, in real time and with bounded complexity, whether the integration must be released or blocked, providing an auditable basis for the safe autonomy of the agent. The effector routes each failure mode to a distinct terminal status, separating process failures (*divergencia_pcp*, production planning and control), classification failures (*erro_classificacao*, computer-vision team) and integration-infrastructure failures (*erro_decisao*, IT).*

Resumo. *Agentes industriais de visão computacional atuam como elo entre o chão de fábrica e a camada de Enterprise Resource Planning (ERP), tomando decisões operacionais com impacto contábil sobre estoque e consumo de matéria-prima. Este artigo especifica quatro restrições operacionais — três como fórmulas em Lógica Temporal Linear (LTL) e LTL Temporizada (TLTL), uma como restrição estrutural — que um agente de visão deve respeitar ao validar o apontamento de produção na janela de abastecimento de um braço de uma máquina de rotomoldagem multibraço. A partir delas, sintetizam-se autômatos de monitoramento via a biblioteca Spot, compõe-se um monitor por produto*

*sincronizado e enuncia-se uma proposição de corretude composicional sob as semânticas ω -regular e LTL_3 de três valores. Três componentes são formalmente especificados: o monitor composto; um mes-bridge pré-processador que materializa o multiconjunto observado e emite eventos de correspondência ou divergência; e o gate de integração MES $\rightarrow$ ERP. O sinal div_i é evento derivado, materializado pelo mes-bridge a partir da comparação multiconjunto e de exceções estruturais do ciclo; e os sumidouros de violação dos monitores são promovidos a eventos sintéticos de diagnóstico — interface monitor-output-as-event que mantém o monitor declarativo e expõe a camada arquitetural de cada falha. O monitor decide em tempo real, com complexidade limitada, se a integração deve ser liberada ou bloqueada, e o efetor roteia cada modo de falha a um status terminal distinto, separando falhas de processo (*divergencia_pcp*, Planejamento e Controle da Produção), de classificação (*erro_classificacao*, visão computacional) e de infraestrutura de integração (*erro_decisao*, TI).*

Keywords: runtime verification; LTL; TLTL; computer vision agents; manufacturing; MES/ERP integration; formal monitoring; compositional monitoring.

Palavras-chave: verificação em tempo de execução; LTL; TLTL; agentes de visão computacional; manufatura; integração MES/ERP; monitoramento formal; monitoramento composicional.

1. Introdução

A introdução de agentes baseados em aprendizado profundo na operação industrial deslocou a fronteira do que se entende por autonomia em manufatura. Em sistemas ciber-físicos de produção, agentes de visão computacional passam a executar tarefas antes confiadas a operadores humanos: identificar peças, contabilizar volumes, validar o cumprimento de Ordens de Produção. Quando a decisão desses agentes alimenta diretamente sistemas transacionais — em particular o ERP, responsável por movimentar saldo de estoque e consumir matéria-prima — a confiabilidade da inferência deixa de ser questão estritamente técnica e passa a constituir requisito de segurança operacional.

Este artigo aborda esse problema em um caso concreto: o apontamento de produção em uma máquina de rotomoldagem multibraço. Na operação instrumentada de referência, o apontamento é gerado pelo *Programmable Logic Controller* (PLC) no instante em que o braço corrente atinge o posto de abastecimento e é encaminhado ao *Manufacturing Execution System* (MES) [Jeon et al. 2017]. A inovação arquitetural proposta neste trabalho consiste em interpor, entre a recepção do apontamento pelo MES e sua integração ao *Enterprise Resource Planning* (ERP), um estado de ciclo de vida adicional — denominado *pendente_verificacao* — durante o qual o apontamento permanece condicionado ao veredito de um monitor formal em tempo real. Sobre o posto de desmoldagem opera um agente de visão computacional que reconstrói, a partir de um *stream* de imagens, o multiconjunto de peças efetivamente desmoldadas na janela de abastecimento $[\tau_a, \tau_b]$. Esse multiconjunto observado, M_{obs} , é comparado ao multiconjunto declarado M_{dec} pelas Ordens de Produção associadas ao apontamento. O monitor, em conjunto com o efetor (*gate*), conduz o apontamento a um de quatro status terminais. Se o multiconjunto observado coincide com o declarado, emite-se correspondência

(match_i), que transita o apontamento ao estado `liberado_integracao` e libera a integração nativa MES→ERP. Se há divergência de multiconjunto ou exceção estrutural do ciclo, materializa-se div_i e o apontamento transita ao estado `divergencia_pcp`, em que o Planejamento e Controle da Produção (PCP) o trata manualmente via fluxo nativo do MES antes da integração ao ERP — a escalada ao PCP é, nesta versão do sistema, ação determinística do *gate*, e não obrigação verificada por monitor. Os outros dois caminhos discriminam falhas de infraestrutura, e não de processo, pela composição do monitor — por meio das propriedades temporizadas A2 e A3, formalizadas na Seção 4: quando a classificação não chega no prazo T_{cls} (violação de A2), o apontamento transita para `erro_classificacao` e é escalado ao time de visão computacional; quando o mes-bridge não pronuncia veredito (match_i ou div_i) no prazo T_{dec} (violação de A3), o apontamento transita para `erro_decisao` e é escalado à área de TI. Crucialmente, o veredito div_i é **evento derivado fabricado pelo mes-bridge** a partir de dois modos de falha de processo — divergência de multiconjunto e exceções estruturais do ciclo —, mantendo o monitor composto livre de raciocínio probabilístico; e os sumidouros de violação de A2 e A3 são **promovidos a eventos sintéticos** consumidos pelo efetor para roteamento ao status correto, mecanismo cuja corretude é demonstrada na composição. A escolha de derivar div_i e de ancorar o prazo de classificação no evento de encerramento da janela de abastecimento (leave_{ab_i}) — e não em cada evento de retirada individual de molde ($\text{rem}_{i,j}$) — responde a uma característica do regime operacional real: classificações remanescentes podem chegar até T_{cls} após o encerramento da janela, de modo que um prazo amarrado às retiradas individuais seria inadequado. A operação atual corrige relativamente erros já manifestados no ERP; a proposta deste trabalho desloca essa correção para intervenção preventiva ancorada em verificação formal em tempo real.

A pergunta que estrutura o artigo é: como especificar formalmente, em LTL e TLTL, as restrições operacionais que o agente deve respeitar durante a janela de abastecimento, e como sintetizar autômatos finitos de monitoramento que verifiquem essas restrições em tempo real, fornecendo base auditável para a transição do apontamento do estado `pendente_verificacao` para `liberado_integracao` ou `divergencia_pcp`, condicionando assim a integração MES→ERP? A questão se insere no campo de *AI Safety* [Amodei et al. 2016] e instancia o paradigma de verificação de restrições em agentes via autômatos de monitoramento.

A contribuição é tripla. **Teórica:** especificação formal de quatro propriedades operacionais (três fórmulas LTL/TLTL e a restrição estrutural A5); síntese explícita dos autômatos de monitoramento correspondentes; ancoragem do prazo de classificação em leave_{ab_i} e classificação de A3 como *liveness* temporizada, refletindo o regime de latência do *mes-bridge*; enunciado de uma proposição de corretude composicional para o monitor agregado sob as semânticas ω -regular e LTL₃; análise de complexidade. O núcleo formal instancia maquinário consolidado de *runtime verification* (produto sincronizado, monitores de *safety* e de *bounded liveness*); a originalidade reside nos planos arquitetural e metodológico a seguir. **Arquitetural:** definição do protocolo de sinalização (eventos de retirada, classificação, comparação multiconjunto, divergência, escalonamento) e de três componentes formalmente especificados — *monitor composto*, *mes-bridge* (pré-processador que materializa o multiconjunto observado M_{obs} e fabrica os eventos de correspondência e divergência) e *gate* MES↔ERP, este último realizado como transição de status no ciclo de vida do apontamento de produção dentro do MES, governada pelo ve-

redito do monitor composto, mediando a integração nativa MES→ERP — entidades hoje inexistentes na infraestrutura industrial considerada. **Metodológica:** disponibilização de um artefato de referência (emulador) que implementa e executa a especificação formal sobre os 15 cenários derivados do contexto operacional e a verifica por *property-based testing* sobre traços gerados aleatoriamente (cf. §5).¹

A Figura 1 sintetiza a arquitetura proposta. O diagrama exhibe o caminho do apontamento de produção, originado no PLC, persistido no MES no estado *pendente_verificacao*, verificado pelo monitor composto a partir do *stream* de visão computacional — com mediação do *mes-bridge* sobre M_{dec} — e finalizado por uma transição de status governada pelo veredito do monitor, com dois destinos possíveis: *liberado_integracao*, que prossegue à integração nativa MES→ERP, ou *divergencia_pcp*, que abre fila de tratativa manual ao PCP no próprio MES antes da integração. Em sombreamento estão os três componentes formalmente especificados: o monitor composto, o *mes-bridge* e o *gate* MES↔ERP — este último realizado como a lógica de transição de status do apontamento.

¹O emulador associado a este trabalho, em Haskell, é fiel à especificação aqui apresentada e cobre os 15 cenários derivados do contexto operacional descrito em §3. O artefato será depositado em repositório público, com DOI, na versão de publicação.

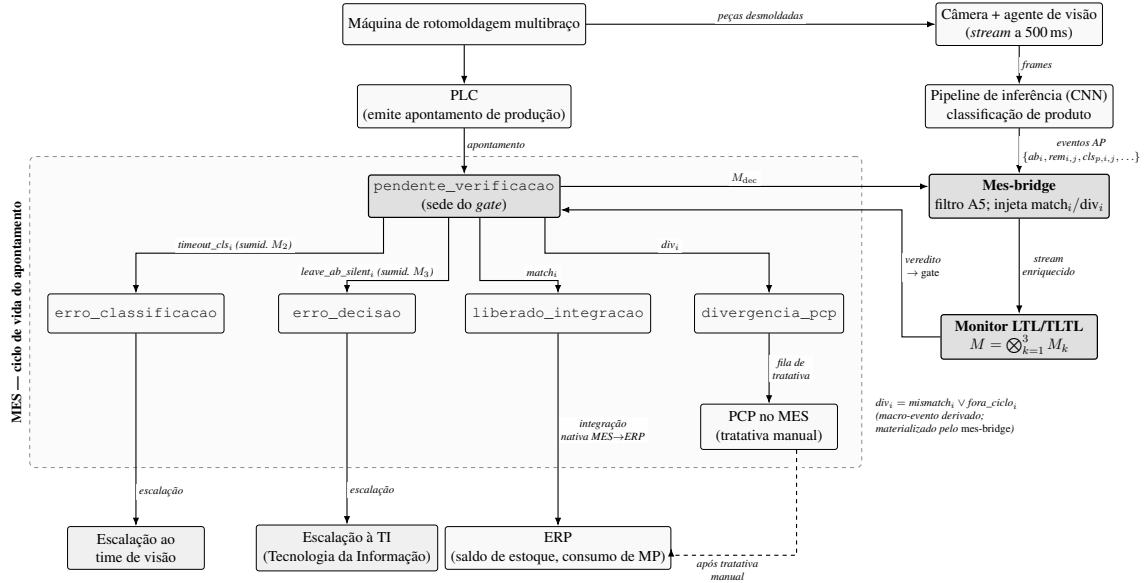


Figura 1. Arquitetura proposta. O apontamento de produção, originado no PLC, persiste no MES no estado `pendente_verificacao` durante a janela de abastecimento $[\tau_a, \tau_b]$. Em paralelo, o agente de visão alimenta o monitor composto via *pipeline CNN*; o *mes-bridge* recebe M_{dec} do apontamento pendente, materializa M_{obs} sob o filtro A5 e injeta $match_i$ ou o macro-evento derivado $div_i = mismatch_i \vee fora_ciclo_i$ no fluxo consumido pelo monitor. O veredito do monitor é devolvido ao MES, atuando como *gate*, e dispara a transição do apontamento para um de quatro status terminais, segundo a causa-raiz. Em `liberado_integracao` (caminho feliz, com $match_i$ dentro de T_{dec} após $leave_ab_i$) o apontamento prossegue à integração nativa MES→ERP. Em `divergencia_pcp` — divergência de processo, escalada deterministicamente a partir de div_i — o apontamento é encaminhado à fila do PCP no MES para tratativa manual antes da integração; a seta tracejada indica o retorno pós-tratativa ao ERP. Em `erro_classificacao` — falha de classificação, sinalizada pelo evento $timeout_cls_i$ que percorre o sumidouro de M_2 — o apontamento é escalado ao time de visão. Em `erro_decisao` — ausência de decisão tempestiva do *mes-bridge*, sinalizada pelo evento $leave_ab_silent_i$ que percorre o sumidouro de M_3 — o apontamento é escalado à TI (Tecnologia da Informação). A figura distingue o *monitor declarativo* (à direita, que apenas observa e emite o veredito) do *gate operacional* (a lógica de transição de status no MES, que consome o veredito e efetua o roteamento). O contêiner tracejado delimita o ciclo de vida do apontamento dentro do MES, englobando os quatro status terminais e o passo de tratativa manual no PCP; os destinos de escalção (time de visão e TI) ficam fora do MES. O *gate MES↔ERP* é realizado como essa lógica de transição de status, não como bloco externo. Os três componentes formalmente especificados — *mes-bridge*, *monitor composto* ($M = \bigotimes_{k=1}^3 M_k$) e o estado `pendente_verificacao` (sede do *gate*) — aparecem em sombreado.

O artigo está organizado da seguinte forma. A Seção 2 fundamenta LTL, TLTL, autômatos de Büchi e *runtime verification*. A Seção 3 formaliza o modelo do agente e a função arquitetural do *mes-bridge*. A Seção 4 enuncia as quatro propriedades. A Seção 5 sintetiza e compõe os autômatos, enuncia a corretude composicional e detalha o *gate* de decisão. A Seção 6 discute extensões e a Seção 7 conclui.

2. Fundamentação Teórica

2.1. Lógica Temporal Linear (LTL)

Lógica Temporal Linear (LTL), introduzida por Pnueli [Pnueli 1977], estende a lógica proposicional com operadores temporais X (próximo), U (até), F (eventualmente) e G (globalmente), interpretados sobre traços ω -infinitos $\sigma \in (2^{AP})^\omega$. Denotando por $\sigma[i..] = \sigma_i\sigma_{i+1}\dots$ o sufixo de σ a partir da posição i , a semântica usual é: $\sigma \models G\varphi$ sse $\sigma[i..] \models \varphi$ para todo $i \geq 0$; $\sigma \models F\varphi$ sse $\sigma[i..] \models \varphi$ para algum $i \geq 0$; $\sigma \models X\varphi$ sse $\sigma[1..] \models \varphi$; $\sigma \models \varphi U \psi$ sse existe j com $\sigma[j..] \models \psi$ e $\sigma[i..] \models \varphi$ para $0 \leq i < j$. As fórmulas dividem-se em *safety* (algo ruim nunca acontece) e *liveness* (algo bom acaba acontecendo) [Manna and Pnueli 1992, Lamport 1977, Alpern and Schneider 1985].

2.2. LTL Temporizada (TLTL)

LTL clássica é qualitativa quanto ao tempo: garante a ocorrência futura de um evento, mas não impõe limite cronológico para sua materialização. Em sistemas industriais, no entanto, propriedades de *liveness* sem prazo são operacionalmente não-monitoráveis sobre prefixos finitos. Para superar essa limitação, recorre-se a LTL Temporizada (TLTL). Adota-se a variante de TLTL definida por [Bauer et al. 2011], baseada nos autômatos temporizados de [Alur and Dill 1994]. Variantes alternativas (MTL [Koymans 1990], MITL [Alur, Feder and Henzinger 1996], TPTL [Alur and Henzinger 1994]) diferem quanto à decidibilidade dos respectivos problemas de *model checking* [Baier and Katoen 2008] — uma comparação detalhada está fora do escopo deste artigo. Em TLTL, os operadores temporais admitem subscritos com intervalos cronológicos: $F_{[0,T]}\varphi$ exige que φ se verifique em algum instante do intervalo $[0, T]$ a partir da posição corrente do traço. O traço é enriquecido com *timestamps* e o monitor pode declarar violação ao expirar T sem a ocorrência do evento esperado.

2.3. Autômatos e Runtime Verification

Autômatos de Büchi [Büchi 1962] aceitam palavras ω -infinitas pela visita infinita a estados aceitantes. A correspondência $LTL \rightarrow \text{Büchi}$ de Vardi e Wolper [Vardi and Wolper 1986] fundamenta a abordagem automata-teórica: para toda fórmula LTL φ existe um autômato $\mathcal{A}_\varphi$ tal que $L(\mathcal{A}_\varphi)$ coincide com o conjunto de traços que satisfazem φ ; para *safety*, basta um autômato finito tradicional, em que a violação corresponde à entrada em sumidouro sem transição válida.

A verificação formal clássica via *model checking* — exemplificada por ferramentas consolidadas como Spin [Holzmann 2003] e NuSMV [Cimatti et al. 2002] — verifica exhaustivamente, *a priori*, se um modelo do sistema satisfaz uma propriedade temporal expressa em LTL ou em lógicas correlatas. Em sistemas ciber-físicos (CPS), contudo, essa abordagem encontra limites estruturais: o sistema interage com o mundo físico por meio de sensores — no presente trabalho, especificamente via visão computacional —, cujo comportamento é dificilmente modelável de forma exata *a priori*, e o espaço de estados associado torna a verificação exhaustiva inviável.

Runtime verification [Leucker and Schallhart 2009] sintetiza, a partir de uma especificação formal, monitores que decidem *online* se cada prefixo do traço observado é compatível com a especificação. Bauer et al. [Bauer et al. 2011] estabeleceram o aparato

canônico para LTL e TLTL, introduzindo a semântica LTL_3 de três valores $\{\top, \perp, ?\}$ (verdadeiro, falso, inconclusivo). Para *safety*, Havelund e Roşu [Havelund and Roşu 2004] descrevem monitoramento com complexidade $O(1)$ amortizada por evento. A síntese explícita é operacionalizada pela biblioteca Spot [Duret-Lutz et al. 2016].

A semântica LTL_3 distingue dois vereditos operacionalmente relevantes neste trabalho. O *veredito de stream*, computado após cada prefixo finito $\sigma^n = \sigma_0\sigma_1 \cdots \sigma_{n-1}$, classifica a especificação como $\top$ (todo prolongamento $\sigma^n \cdot w$, com $w \in (2^{AP})^\omega$, satisfaz φ), $\perp$ (nenhum prolongamento satisfaz φ) ou $?$ (existem prolongamentos satisfazendo e prolongamentos violando φ). O *veredito terminal*, computado ao final do traço observado, colapsa $?$ em $\perp$ quando obrigações de *liveness* permanecem pendentes sem possibilidade de resolução. Operacionalmente, $?$ durante o *stream* significa “ainda pode ser resgatada”; o colapso em $\perp$ ao término marca a obrigação como definitivamente violada. Esta distinção é essencial para o *gate* (§5.4), pois decisões de bloqueio podem ser emitidas tanto na visita a um sumidouro de violação durante o *stream* quanto na transição $? \rightarrow \perp$ no instante terminal.

Conceitualmente, o *gate* $MES \leftrightarrow ERP$ proposto neste trabalho instancia os paradigmas de *enforcement* e de *shielding*, adaptados ao contexto de agentes de visão computacional em manufatura discreta — posicionamento detalhado na §2.4. Uma sistematização recente do campo de *runtime verification* encontra-se em [Bartocci et al. 2018].

Mais próximo deste trabalho está o monitoramento de componentes de aprendizado: técnicas que observam o *interior* da rede neural — padrões de ativação de neurônios [Cheng et al. 2019], abstrações por caixas sobre o espaço latente [Henzinger et al. 2020] e monitoramento ativo de novidade/*out-of-distribution* [Lukina et al. 2021] — para detectar entradas fora da distribuição de treino. O presente trabalho é complementar e opera em outra camada: a CNN é tratada como caixa-preta e o *gate* monitora as *proposições de saída* do agente (retirada, classificação, correspondência multiconjunto) contra restrições operacionais temporizadas, em vez de inspecionar ativações internas. Monitorar a *decisão* do agente — e não sua representação interna — delimita a contribuição frente a essa linha. O *delta* específico não está no maquinário de *runtime verification* empregado (produto sincronizado, monitores de *safety* e de *bounded liveness*, todos consolidados), mas em três elementos arquiteturais e metodológicos: o *mes-bridge* que fabrica os vereditos de coerência, a interface *monitor-output-as-event* que promove sumidouros a eventos sintéticos, e a separação de modos de falha em status MES com causa-raiz auditável.

A Tabela 1 sintetiza o posicionamento deste trabalho frente a abordagens correlatas de *runtime verification* e sistemas ciber-físicos de manufatura.

Tabela 1. Posicionamento da presente proposta frente a abordagens correlatas de *runtime verification* (RV) e sistemas ciber-físicos de manufatura. As colunas binárias usam ✓ para “sim” e × para “não”; “—” indica não-aplicável (trabalho fora do escopo de RV ou de domínio industrial). A última linha sintetiza a contribuição arquitetural-teórica deste artigo.

Trabalho	Lógica suportada	Domínio	Composic.	Visão?
[Havelund & Roşu 2004]	LTL (safety)	genérico	×	×
[Leucker & Schallhart 2009]	LTL (survey)	genérico	—	×
[Bauer <i>et al.</i> 2011]	LTL ₃ , TLTL	genérico	implícita	×
[Duret-Lutz <i>et al.</i> 2016]	LTL, ω -autômatos	ferramenta (Spot)	✓	×
[Pinisetty <i>et al.</i> 2017]	LTL, MTL	enforcement / CPS	implícita	×
[Junges <i>et al.</i> 2021]	PCTL, ω -aut.	shielding / POMDP	—	×
[Bartocci <i>et al.</i> 2018]	LTL, MTL, STL	RV (survey)	—	×
[D’Angelo <i>et al.</i> 2005]	LTL sobre <i>streams</i>	<i>stream</i> RV / síncrono	implícita	×
[Convent <i>et al.</i> 2018]	TeSSLa (<i>timestamps</i>)	<i>stream</i> RV / CPS	implícita	×
[Schneider 2000]	autômatos de segurança	<i>enforcement</i>	—	×
[Alshiekh <i>et al.</i> 2018]	lógica temporal	<i>shielding</i> / RL	—	×
Este trabalho	LTL + TLTL	manufatura / visão CPS (rotomoldagem)	explícita (Prop. 1)	✓

2.4. Posicionamento frente a vertentes correlatas

As três vertentes a seguir — *stream RV*, *enforcement/shielding* e monitoramento temporizado — são as correlatas centrais ao mecanismo proposto; para cada uma, explicita-se o que o presente trabalho *instancia*, *estende* ou *inova*, sem reivindicar originalidade no maquinário consolidado de *runtime verification*.

2.4.1. Stream RV e Transdução de Eventos

A abordagem de *runtime verification* para sistemas com dinâmica de eventos foi intensificada por motores de transdução de *streams*. LOLA [D’Angelo et al. 2005] estabeleceu o paradigma de linguagem de especificação para monitoramento síncrono, computando *streams* de saída a partir de *streams* de entrada por equações recursivas e operadores de deslocamento temporal. TeSSLa [Convent et al. 2018] estende a abordagem para fluxos com *timestamps* nativos, integrando especificação declarativa e rastreamento de tempo explícito — relevante para sistemas ciber-físicos que requerem sincronização com eventos de hardware.

O *mes-bridge* proposto pode ser posicionado como uma transdução de *stream*: materializa o multiconjunto observado M_{obs} por SKU a partir do fluxo de classificações $\text{cls}_{p,i,j}$ e injeta os eventos derivados match_i e div_i como saída. Contudo, o delta não reside no maquinário de transdução — LOLA e TeSSLa já forneceriam os componentes sintáticos —, mas em três aspectos: (i) *semântica de coerência multiconjunto por SKU*, que compara multiplicidades e não apenas sequências de eventos, operação não padrão em LOLA/TeSSLa; (ii) *posicionamento arquitetural entre agente de visão e MES*, em que

o transdutor executa validação pré-integração; (iii) *roteamento determinístico de modos de falha* a estados do MES com causa-raiz auditável. A transdução é guiada por requisitos operacionais, não apenas por computação simbólica de saída.

Maler e Nickovic [Maler and Nickovic 2004] fundamentaram o monitoramento de propriedades métricas e temporizadas; este trabalho aproveita essa base: as propriedades A2 e A3, em TLTL, codificam os prazos operacionais (T_{cls} , T_{dec}). A monitorabilidade em prefixo finito de *bounded liveness* é garantida pela teoria de Bauer, Leucker e Schallhart [Bauer, Leucker and Schallhart 2007], que estabelece quando um veredito definitivo em prefixo finito é possível — condição que A2 e A3 satisfazem por ancoragem em `leave_abi` e prazos finitos. Pnueli e Zaks [Pnueli and Zaks 2006] formalizaram abordagem complementar via *testers* como transdutores composicionais.

2.4.2. Enforcement e Shielding em Integração de Sistemas

O paradigma de *enforcement monitors* tem raízes em Schneider [Schneider 2000], que introduziu as *security automata* e o conceito de execução monitorada. Falcone, Fernandez e Mounier [Falcone, Fernandez and Mounier 2012] expandiram o arcabouço com uma taxonomia que responde “quais propriedades podem ser verificadas e enforçadas em tempo de execução?”: *safety* é naturalmente enforceable, enquanto *liveness* sem prazo não é monitorável em prefixo finito. *Shielding*, conceituado em Alshiekh et al. [Alshiekh et al. 2018], sintetiza *shields* a partir de especificação temporal, garantindo que um agente nunca viole uma propriedade de segurança.

O *gate* MES $\leftrightarrow$ ERP instancia elementos desse paradigma, com diferenças arquiteturais: enquanto enforcement clássico (Schneider, Falcone) reescreve ou bloqueia *ações de baixo nível* (e.g. variáveis de controle em Pinisetty et al. [Pinisetty et al. 2017]), o gate decide sobre *transições de status em camada de negócio* (o apontamento entre estados do MES). O veredito roteia a proposição de saída do agente a quatro destinos (`liberado_integracao`, `divergencia_pcp`, `erro_classificacao`, `erro_decisao`), em vez de editar o fluxo de controle. Diferencia-se também do *shielding* probabilístico (POMDP, Junges et al. [Junges et al. 2021]): o gate é determinístico e repousa em vereditos binários (match/divergência) de comparação multiconjunto verificável, não em estimação de estado parcialmente observado. O enforcement é, aqui, uma forma de *orquestração de estado* na integração de sistemas. Disso decorrem dois ganhos: (i) *auditabilidade* (cada transição é rastreável ao modo de falha que a disparou); e (ii) *compatibilidade com a infraestrutura existente* (sem modificar o MES ou o controlador lógico).

2.4.3. Monitoramento Temporizado e Monitorabilidade

Bauer, Leucker e Schallhart [Bauer, Leucker and Schallhart 2007] consolidaram a monitorabilidade em prefixo finito para LTL: *safety* é sempre monitorável e *bounded liveness* também o é, desde que a violação tenha testemunha de tamanho finito. Bauer et al. [Bauer et al. 2011] refinam a semântica de três valores LTL_3 ($\top$, $\perp$, $?$) para TLTL com relógios; essa semântica é operacionalmente crítica aqui: $?$ durante o fluxo significa “ainda pode ser resgatada”, e a transição para $\perp$ ao término do prazo marca violação de-

finitiva — distinção que o monitor composto explora para separar bloqueios antecipados (entrada em sumidouro) de bloqueios por expiração ($? \rightarrow \perp$).

A composição de propriedades temporizadas pelo ínfimo $\sqcap$ (Proposição 2 deste trabalho) herda monitorabilidade: se cada componente é monitorável em seu prazo, a composição sincronizada também o é. Pnueli e Zaks [Pnueli and Zaks 2006] formalizaram a correspondência entre *testers* e monitoramento em prefixo finito, enfatizando que decisões locais se agrupam via produto sincronizado — padrão que o monitor composto segue, com a corretude composicional garantida pelas Proposições 1 e 2 (Apêndice C). Nenhum desses elementos é novidade isolada; sua integração na arquitetura (mes-bridge + monitor composto + gate) realiza uma orquestração formal e auditável, especializada à manufatura discreta.

3. Modelo Formal do Agente

3.1. Cenário operacional

Considera-se uma máquina de rotomoldagem com três braços operando em ciclo contínuo, cada braço percorrendo sequencialmente os estágios abastecimento $\rightarrow$ forno $\rightarrow$ resfriamento $\rightarrow$ abastecimento. O artigo restringe a análise ao estágio de abastecimento, intervalo durante o qual o operador desmolda peça a peça os moldes do braço corrente. Um braço comporta um número finito e variável de moldes — limitado por restrições físicas e de engenharia, como a envergadura do braço, as dimensões de cada molde e o balanceamento de massa do conjunto — e pode envolver uma ou mais Ordens de Produção simultaneamente, situação típica em ambientes *job-shop* (produção sob encomenda, com roteiros variados) de manufatura discreta [Leitão 2009, Monostori et al. 2006].

Não há sinais discretos provenientes do *Programmable Logic Controller* (PLC) ou do MES atual que delimitem eventos no nível de molde individual. Há, porém, um sinal canônico nesse caminho: o próprio apontamento de produção, emitido pelo PLC ao MES no instante da chegada do braço ao posto de abastecimento e persistido no MES no estado `pendente_verificacao` durante a janela. O apontamento, no entanto, não decompõe a janela em eventos de molde individual — esta granularidade é exclusiva do agente de visão. Um agente de visão computacional, posicionado sobre o posto de desmoldagem, recebe um *stream* de imagens à cadência de 500 ms e é o responsável por reconstruir a sequência canônica de estados por molde: *molde fechado* $\rightarrow$ *produto identificado* $\rightarrow$ *molde vazio* $\rightarrow$ *molde saiu da posição* $\rightarrow$ *próximo molde*. Esta configuração instancia o padrão de agente em *cyber-physical production systems* descrito por [Leitão et al. 2016, Cardin 2019, Törngren et al. 2021].

3.2. Sinais como abstrações do agente

Sejam B , M e P os conjuntos finitos de braços, de posições de molde por braço e de produtos admissíveis, respectivamente. A janela de abastecimento do braço $i \in B$ é o intervalo $[\tau_a^i, \tau_b^i]$, em que τ_a^i marca a chegada do braço i ao posto de desmoldagem e τ_b^i sua partida. Durante a janela, os moldes $j \in M$ do braço são processados sequencialmente, gerando, para cada molde, um evento de retirada seguido de um evento de classificação. Define-se o conjunto de proposições atômicas

$$\begin{aligned} AP = & \{ \text{ab}_i, \text{leave_ab}_i, \text{match}_i, \text{div}_i : i \in B \} \\ & \cup \{ \text{rem}_{i,j}, \text{cls}_{p,i,j} : i \in B, j \in M, p \in P \}, \end{aligned}$$

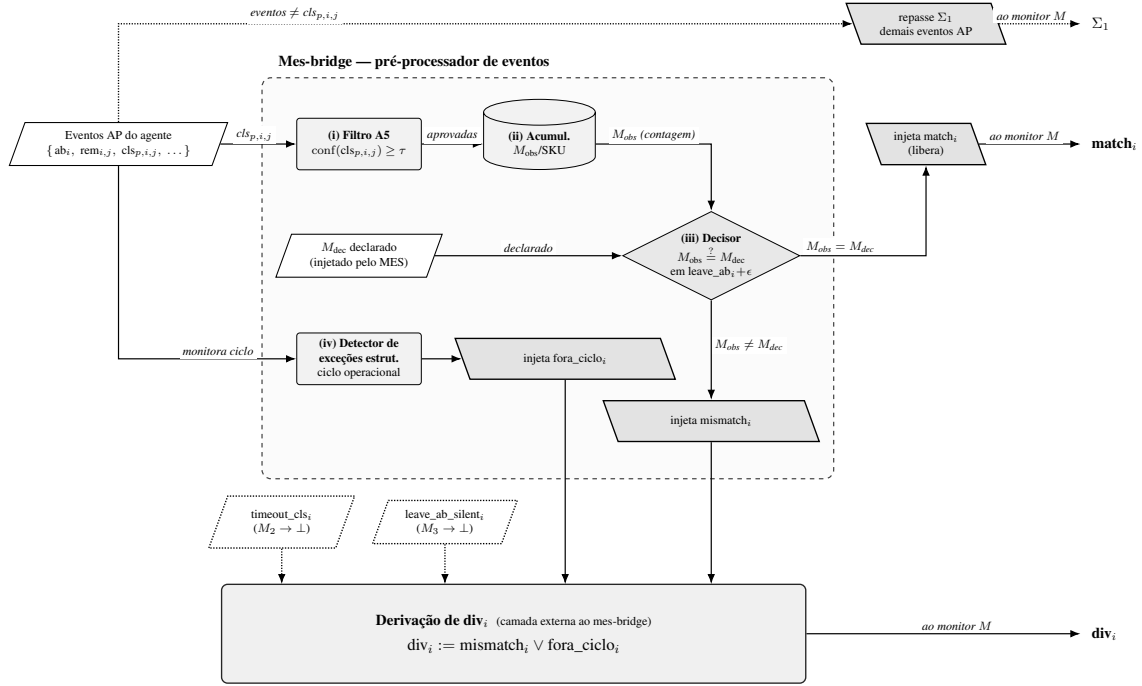
em que ab_i indica que o braço i está na janela de abastecimento; $rem_{i,j}$ sinaliza a retirada da peça do molde j durante a janela do braço i ; $cls_{p,i,j}$ indica a classificação dessa peça como produto $p \in P$; $leave_ab_i$ marca o encerramento da janela do braço i ; $match_i$ e div_i representam, respectivamente, a conformidade ou a divergência do multiconjunto $\{cls_{p,i,j}\}_{j \in M, p \in P}$ acumulado em relação ao esperado, sendo avaliados uma única vez ao final da janela. A cardinalidade do conjunto é

$$|AP| = 4|B| + |B| \cdot |M| \cdot (1 + |P|),$$

e, consequentemente, $|\Sigma| = 2^{|AP|}$. A proposição div_i , embora listada em AP por ser efetivamente consumida pelas propriedades e pelo *gate*, é um *macro-evento derivado* (§3.4); seus dois sub-eventos constituintes ($mismatch_i$ e $fora_ciclo_i$) pertencem ao fluxo enriquecido produzido pelo mes-bridge, não a AP , de modo que a cardinalidade acima permanece válida.

Crucialmente, **nenhum dos sinais em AP existe como evento discreto proveniente da máquina, do PLC ou do MES atual**. São abstrações produzidas pelo próprio agente de visão a partir do *stream* de imagens, com uma ressalva importante (formalizada a seguir): $match_i$ e div_i não são inferidos diretamente pela rede neural, mas *fabricados* por uma camada arquitetural intermediária — o *mes-bridge* — a partir da comparação entre o multiconjunto observado e o declarado pelo MES. Esta propriedade tem consequência teórica relevante: o agente é simultaneamente o gerador da maioria dos eventos e o sujeito das restrições sobre eles, situação que reforça a necessidade de verificação formal interna ao agente, já que não há observador externo independente para arbitrar discrepâncias [Amodei et al. 2016].

O mes-bridge como camada arquitetural formalmente especificada. Entre a saída do agente de visão e a entrada do monitor composto interpõe-se um pré-processador estacionário, denominado *mes-bridge*, com três responsabilidades: (i) aplicar o filtro semântico da propriedade A5 (§4) sobre as classificações $cls_{p,i,j}$; (ii) acumular, durante a janela estendida $[\tau_a^i, \tau_b^i + T_{cls}]$ (a janela acrescida do orçamento de classificação, §4), o multiconjunto observado M_{obs} por SKU (*Stock Keeping Unit*); e (iii) após $leave_ab_i$, esgotado o orçamento T_{cls} de classificações residuais, comparar M_{obs} com o multiconjunto declarado M_{dec} proveniente do MES e injetar, no fluxo de eventos consumido pelo monitor e dentro do prazo T_{dec} de A3, $match_i$ se $M_{obs} = M_{dec}$ ou div_i caso contrário. Note-se que M_{dec} não é uma constante global, mas atributo do apontamento de produção corrente, persistido no MES no estado *pendente_verificacao* e injetado pelo MES no *mes-bridge* no instante de abertura da janela $[\tau_a, \tau_b]$. A Figura 2 apresenta sua estrutura interna. Esta separação responde explicitamente à questão “quem fabrica $match_i$ e div_i ?” e isola, em uma camada arquitetural autônoma, a transformação estatística (filtro de confiança) e a operação multiconjunto, deixando o monitor composto livre de raciocínio probabilístico.



Linha sólida: fluxo de eventos/dados no mes-bridge. Linha pontilhada: repasse Σ_1 e sub-eventos sintéticos ($M_2, M_3 \rightarrow$ evento). Contorno tracejado: fronteira de zona arquitetural (mes-bridge / derivação de div_i).

Figura 2. Vista interna do mes-bridge. A caixa tracejada delimita a camada formalmente especificada do mes-bridge, com quatro estágios funcionais numerados: (i) Filtro A5, que aceita apenas classificações $cls_{p,i,j}$ com confiança $\geq \tau$; (ii) Acumulador M_{obs} , que contabiliza por SKU as classificações aprovadas; (iii) Decisor multiconjunto, ativado em $leave_ab_i + \epsilon$, que compara M_{obs} ao multiconjunto declarado M_{dec} recebido do apontamento de produção no estado `pendente_verificacao` e injeta $match_i$ (igualdade, libera) ou $mismatch_i$ (diferença); e (iv) Detector de exceções estruturais, que monitora o ciclo operacional a partir dos eventos AP e injeta $fora_ciclo_i$ ao identificar quebras de ciclo. Os demais eventos AP atravessam o mes-bridge sem modificação (repasso Σ_1 , linha pontilhada no topo). A caixa sólida inferior, externa ao mes-bridge, materializa div_i como *evento derivado*: $div_i := mismatch_i \vee fora_ciclo_i$. Os sub-eventos sintéticos `timeout_cls_i` e `leave_ab_silent_i` são promovidos pelos sub-monitores M_2 e M_3 , respectivamente, ao alcançarem seus sumidouros de violação (cf. §3.4). As três saídas que alimentam o monitor composto M — $match_i$, div_i e o repasse Σ_1 — são exibidas à direita.

3.3. Multiconjunto observado versus declarado

Recorde-se que P é o conjunto finito de SKUs (produtos) admissíveis (§3.2). Define-se o multiconjunto observado $M_{obs} : P \rightarrow \mathbb{N}$ como a função que contabiliza, para cada $p \in P$, o número de proposições $cls_{p,i,j}$ verificadas durante a janela estendida $[\tau_a^i, \tau_b^i + T_{cls}]$, descontados os sinais cuja confiança de classificação não atinge o limiar τ . O multiconjunto declarado $M_{dec} : P \rightarrow \mathbb{N}$ é o agregado de todas as Ordens de Produção atribuídas ao braço i na janela. A proposição $match_i$ é verdadeira, em até T_{dec} após $leave_ab_i$ (propriedade A3, §4), se e somente se $M_{obs}[\tau_a^i, \tau_b^i + T_{cls}] = M_{dec}$, e div_i é verdadeira, no mesmo prazo, caso contrário.

O caso de borda em que ambos os multiconjuntos são vazios merece registro explícito. Se a janela transcorre sem retiradas e o braço não tem Ordens de Produção atri-

buídas (ou as Ordens contemplam apenas posições de molde vazio para balanceamento de massa, sem peças efetivas), então $M_{\text{obs}} = M_{\text{dec}} = \emptyset$ e a igualdade $\emptyset = \emptyset$ se verifica trivialmente. Pela definição acima, match_i é emitido em leave_ab_i ; a propriedade A2 (§4) é satisfeita vacuamente — sua guarda houve_rem_i , verdadeira apenas se ao menos uma retirada $\text{rem}_{i,j}$ ocorreu na janela, é falsa — e A3 é satisfeita pela emissão de match_i no prazo, sem tratamento particular. Esta convenção evita que janelas operacionalmente inertes sejam interpretadas como violação de *safety* ou *liveness*.

A Figura 3 ilustra a ordenação temporal das proposições atômicas dentro da janela $[\tau_a, \tau_b]$, incluindo as restrições temporizadas T_{cls} e T_{dec} que serão formalizadas na próxima seção.

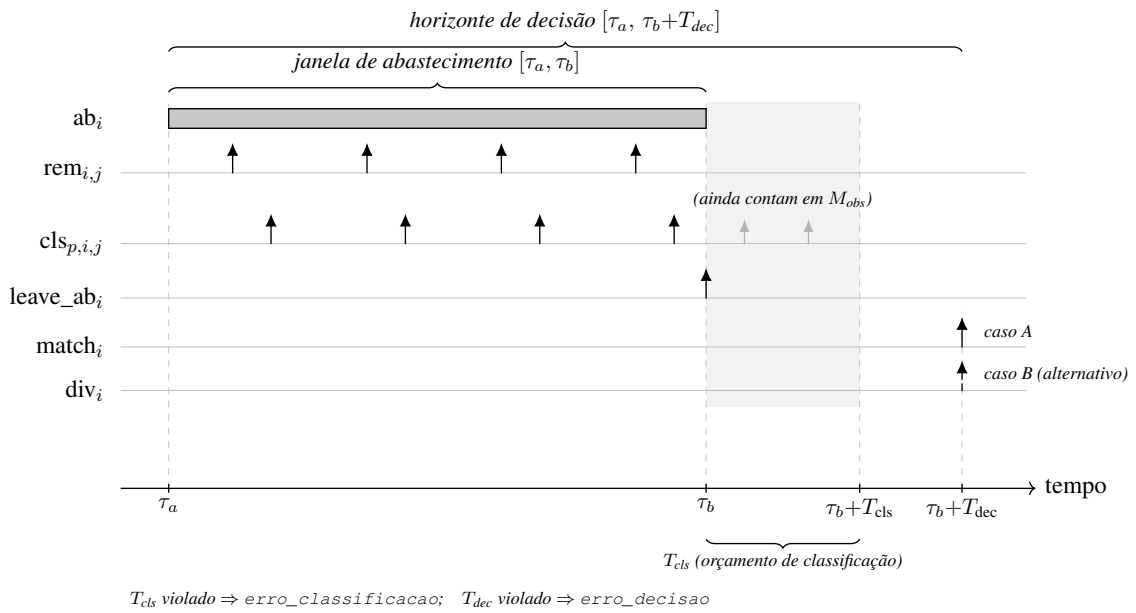


Figura 3. Linha do tempo da janela de abastecimento. A faixa cinza-escura na lane ab_i marca a *janela física* $[\tau_a, \tau_b]$ em que ab_i é verdadeira; as setas sólidas indicam as ocorrências de $\text{rem}_{i,j}$ (retiradas) e $\text{cls}_{p,i,j}$ (classificações) dentro da janela. A faixa cinza-clara $[\tau_b, \tau_b + T_{\text{cls}}]$ representa o *orçamento de classificação*: as classificações residuais (setas cinzas após τ_b) contam para M_{obs} , pois A2 é ancorada em leave_ab_i e verifica *existência agregada* de classificação válida no intervalo $[\tau_a, \tau_b + T_{\text{cls}}]$. O veredito do mes-bridge é emitido em $\tau_b + T_{\text{dec}}$, com $T_{\text{dec}} \geq T_{\text{cls}}$, como match_i (caso A, seta sólida) ou, alternativamente, div_i (caso B, seta tracejada) — prazo capturado por A3, propriedade de *liveness* temporizada. Os marcos terminais T_{cls} e T_{dec} correspondem, em caso de violação, aos status *erro_classificacao* (orçamento de classificação expirado) e *erro_decisao* (horizonte de decisão expirado), respectivamente. O intervalo $[\tau_a, \tau_b + T_{\text{dec}}]$ constitui o *horizonte de decisão*, distinto da janela física.

3.4. Eventos derivados e exceções estruturais

O alfabeto de proposições atômicas AP apresentado em §3.2 inclui div_i como proposição. div_i é *evento derivado*, definido como disjunção formal de dois modos de falha de processo, ambos materializados pelo mes-bridge:

$$\text{div}_i := \text{mismatch}_i \vee \text{fora_ciclo}_i,$$

em que:

- mismatch_i é emitido pelo mes-bridge em $\tau_b^i + T_{\text{cls}} + \delta_{\text{mb}}$ quando $M_{\text{obs}}[\tau_a^i, \tau_b^i + T_{\text{cls}}] \neq M_{\text{dec}}$, em que $\delta_{\text{mb}} \leq T_{\text{dec}} - T_{\text{cls}}$ é a latência de comparação do mes-bridge. Materializa o caso “Ordem de Produção errada” ou “contagem por SKU divergente”.
- fora_ciclo_i é emitido pelo mes-bridge ao detectar exceções estruturais do ciclo operacional. A lista corrente, capturada pelo formalismo de exceções do MES, inclui: (a) tentativa de match_i após $\tau_b^i + T_{\text{dec}}$; (b) duplicação de leave_ab_i em um mesmo ciclo; (c) ausência total de ab_i acompanhada de eventos $\text{rem}_{i,j}$ ou $\text{cls}_{p,i,j}$ (violação de A1 detectada estruturalmente); (d) inconsistência entre o M_{dec} injetado pelo MES e a OP ativa no apontamento; (e) outras exceções operacionais futuras, plugáveis pela mesma interface.

Da mesma natureza derivada é a guarda houve_rem_i , consumida por A2 (§4): mantida pelo mes-bridge, é verdadeira em leave_ab_i se e somente se ao menos uma retirada $\text{rem}_{i,j}$ ocorreu na janela corrente. Como os sub-eventos de div_i , vive no fluxo enriquecido, sem ser acrescentada a AP .

Além de div_i , a arquitetura emprega dois *eventos sintéticos de diagnóstico*, produzidos pela promoção dos sumidouros de violação dos sub-monitores temporizados a fontes de evento (interface *monitor-output-as-event*):

- timeout_cls_i , emitido quando o sub-monitor M_2 (A2')² alcança seu sumidouro de violação por expiração do relógio T_{cls} sem classificação válida acumulada. Materializa o caso “rede neural não classificou em tempo” e é consumido pelo efector para transitar o apontamento a $\text{erro_classificacao}$, escalando-o ao time de visão.
- leave_ab_silent_i , emitido quando o sub-monitor M_3 (A3') alcança seu sumidouro por expiração do relógio T_{dec} sem que o mes-bridge tenha injetado match_i nem div_i . Materializa o caso “mes-bridge mudo” ou “falha do pipeline de comparação multiconjunto” e é consumido pelo efector para transitar o apontamento a erro_decisao , escalando-o à TI.

Diferentemente de mismatch_i e fora_ciclo_i , esses dois eventos *não* constituem div_i : a divergência de multiconjunto e as exceções estruturais são falhas de *processo* encaminhadas ao PCP (status divergencia_pcp), ao passo que a expiração de T_{cls} e a de T_{dec} são falhas de *infraestrutura* encaminhadas, respectivamente, à visão ($\text{erro_classificacao}$) e à TI (erro_decisao).

Cada um desses eventos é produzido por uma fonte arquitetural distinta: mismatch_i e fora_ciclo_i vêm do mes-bridge; timeout_cls_i e leave_ab_silent_i vêm dos sub-monitores M_2 e M_3 “promovidos” a fontes de eventos via a interface *monitor-output-as-event*. Esta construção é detalhada na Figura 2 e na Figura 8.

Esta separação tem três consequências teóricas. Primeira, embora div_i figure em AP como proposição única — contada uma só vez na cardinalidade $|AP|$ de §3.2 —,

²Escreve-se A2' e A3' para os sub-monitores sintetizados M_2 e M_3 das propriedades temporizadas A2 e A3 (§4); a plica marca a forma operacional dessas propriedades consumida pelo monitor composto, com o mesmo conteúdo lógico de A2 e A3.

nem seus dois sub-eventos constituintes (mismatch_i , fora_ciclo_i) nem os eventos sintéticos de diagnóstico (timeout_cls_i , leave_ab_silent_i) são acrescentados a AP : vivem no fluxo enriquecido produzido pelo mes-bridge e pela promoção de sumidouros (formalizado em §5). Tratar div_i como *macro-evento* derivado, portanto, não infla o alfabeto. Segunda, a promoção de sumidouro a evento é uma transdução de estado finito determinística e causal (Apêndice C, Lema 1), de modo que o enriquecimento do fluxo preserva ω -regularidade e a corretude composicional do produto sincronizado $M_1 \otimes M_2 \otimes M_3$ (Proposições 1 e 2) permanece válida. Terceira, a localização da falha — qual evento (mismatch_i , fora_ciclo_i , timeout_cls_i ou leave_ab_silent_i) disparou a transição de status — torna-se atributo auditável do veredito do gate, o que reforça o argumento de *transparência estrutural* explorado em §5.4.

4. Propriedades a Monitorar

Quatro propriedades capturam as restrições operacionais que o agente deve respeitar — A1–A3, em LTL/TLTL, e A5, estrutural; a numeração reserva A4 a uma extensão futura de *bounded liveness* sobre o escalonamento ao PCP, discutida na §6. As fórmulas a seguir adotam a sintaxe aceita pela ferramenta Spot.

A1 (*safety*): $G(\text{rem}_{i,j} \rightarrow \text{ab}_i)$

Toda retirada de peça deve ocorrer dentro da janela de abastecimento. A propriedade é *safety*; o autômato sintetizado tem dois estados, sendo q_0 aceitante com auto-laço sobre as letras que preservam a invariante $\text{rem}_{i,j} \rightarrow \text{ab}_i$, e $q_\perp$ sumidouro de violação alcançado por $\text{rem}_{i,j} \wedge \neg \text{ab}_i$.

Motivação: A1 monitora o agente, não o mundo. À primeira leitura, A1 parece capturar uma invariante física: “o operador está fisicamente no posto de desmoldagem, logo toda retirada ocorre na janela” — e portanto seria trivialmente satisfeita. Esta leitura confunde *determinismo físico* com *correção formal verificável* sobre o traço de eventos. O monitor não observa o mundo: observa as proposições atômicas *produzidas pelo agente de visão* a partir do stream de imagens. O agente pode emitir $\text{rem}_{i,j}$ em três classes de cenário em que ab_i não vale: (i) deriva de modelo na CNN (drift), com falso positivo de retirada entre ciclos; (ii) oclusão parcial da câmera por mão do operador, sombra ou fumaça do forno adjacente, gerando inferência espúria; (iii) intervenção manual com a janela fechada (limpeza, manutenção corretiva), com peça sendo manuseada fora do regime de produção. Em todos os três, o monitor de A1 deve disparar. A1 é, portanto, o *sanity check* operacional do agente: a propriedade “mais óbvia” do conjunto é justamente a que assegura que o agente está operando no regime válido. Sua presença na especificação é metodologicamente essencial e a sua aparente trivialidade é o argumento de auditabilidade, não a sua fragilidade. Operacionalmente, embora A1 vigie o agente de visão, sua violação — uma retirada fora da janela — caracteriza apontamento estruturalmente malformado e é tratada como exceção do ciclo (fora_ciclo_i , §3.4), roteada a divergencia_pcp para tratativa do PCP, independentemente de a causa-raiz residir no agente.

A Figura 4 apresenta o autômato sintetizado para A1.

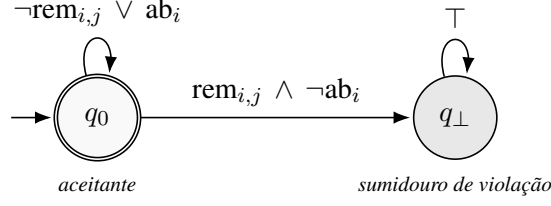


Figura 4. Autômato finito de monitoramento sintetizado para a propriedade A1 ($G(\text{rem}_{i,j} \rightarrow \text{ab}_i)$, *safety*). O estado q_0 é simultaneamente inicial e aceitante; o auto-laço cobre todas as letras que preservam a invariante $\text{rem}_{i,j} \rightarrow \text{ab}_i$. A transição para o estado-sumidouro $q_\perp$ ocorre quando $\text{rem}_{i,j}$ se verifica fora da janela de abastecimento ($\text{rem}_{i,j} \wedge \neg \text{ab}_i$), configurando violação irreversível: nenhuma transição subsequente reabilita a aceitação. O símbolo $\top$ no auto-laço de $q_\perp$ denota qualquer letra do alfabeto $\Sigma = 2^{AP}$. A propriedade é o *sanity check* operacional do agente; sua aparente trivialidade é fundamento da auditabilidade, não fragilidade — ver discussão em §4.1 do texto principal.

A2 **(liveness temporizada):**

$$G\left(\left(\text{leave_ab}_i \wedge \text{houve_rem}_i\right) \rightarrow F_{[0, T_{\text{cls}}]} \bigvee_{j \in M, p \in P} \text{cls}_{p,i,j}^{\geq \tau}\right)$$

Em janelas com ao menos uma retirada, ao encerramento da janela de abastecimento deve existir ao menos uma classificação válida (com confiança $\geq \tau$, A5) acumulada no intervalo $[\tau_a^i, \tau_b^i + T_{\text{cls}}]$. Aqui, $\text{cls}_{p,i,j}^{\geq \tau}$ denota uma classificação que satisfaz o filtro A5 ($\text{cls}_{p,i,j}$ com confiança $\geq \tau$); e houve_rem_i é a guarda — verdadeira em leave_ab_i se e somente se ao menos uma $\text{rem}_{i,j}$ ocorreu na janela $[\tau_a^i, \tau_b^i]$ — mantida pelo mes-bridge, que arma a obrigação de *liveness* apenas quando houve produção a classificar. Em janelas inertes houve_rem_i é falsa, o antecedente da implicação é falso e A2 é satisfeita vacuamente, sem disparar divergência espúria (semântica *opt-in* análoga à de A6, §6).

Escopo: vivacidade, não completude. A2 é o *signal de vida* temporizado do *pipeline* de classificação: seu papel é assegurar que o agente *não está inerte* — que produz ao menos uma classificação válida dentro do prazo T_{cls} —, isolando o modo de falha “classificador parado ou atrasado”, materializado pelo sumidouro de M_2 , que injeta o evento sintético timeout_cls_i (§3.4). A2 verifica *existência*, e não *completude*, por decisão de projeto: a correspondência integral entre o multiconjunto observado e o declarado ($M_{\text{obs}} = M_{\text{dec}}$) é responsabilidade de A3 e do evento mismatch_i emitido pelo mes-bridge, não de A2. As duas propriedades são complementares — A2 garante o *prazo* (o pipeline produz a tempo); A3 e o *mismatch*, o *valor* (a contagem confere). Janelas legitimamente vazias ($M_{\text{dec}} = \emptyset$, sem retiradas) satisfazem A2 *vacuamente* — a guarda houve_rem_i torna o antecedente da implicação falso —, de modo que a ausência de produção a classificar não dispara divergência espúria.

Justificativa da ancoragem. A escolha de ancorar o relógio em leave_ab_i , e não em eventos individuais de retirada $\text{rem}_{i,j}$, deriva de três características do regime operacional real. (i) *Latência variável da inferência*: a CNN do agente opera em paralelo ao *stream* a 500 ms; classificações de uma retirada podem chegar após o agente já ter processado

a próxima retirada, o que tornaria um emparelhamento estrito “ $\text{rem}_{i,j}$ dispara, $\text{cls}_{p,i,j}$ resolve” sensível à ordem em que a CNN libera resultados, não à validade da janela. (ii) *Eventos pós-retirada que alteram a obrigação*: dentro da janela ainda podem ocorrer *abort* de ciclo, dupla retirada do mesmo molde, detecção de molde vazio ou retomada após pausa — qualquer um deles altera o mapeamento entre retiradas e classificações esperadas. (iii) *Ancoragem do mes-bridge*: a comparação $M_{\text{obs}} = M_{\text{dec}}$ realizada pelo mes-bridge ocorre ao final da janela e tem como insumo o multiconjunto acumulado de *todas* as classificações válidas até $\tau_b^i + T_{\text{cls}}$, não o emparelhamento pontual retirada-a-classificação. A formulação A2 reflete diretamente esse insumo arquitetural: o monitor não verifica latência de inferência por retirada individual, mas *existência de pelo menos uma classificação válida acumulada ao final da janela acrescida do orçamento* T_{cls} .

Variante metodológica. Contempla-se, restrita ao regime de diagnóstico interno do agente (e portanto não aplicada ao *gate* de integração $\text{MES} \leftrightarrow \text{ERP}$), a formulação alternativa

$$\text{A2}_{\text{lat}} : G \left(\text{rem}_{i,j} \rightarrow F_{[0, T_{\text{cls}}]} \bigvee_{p \in P} \text{cls}_{p,i,j} \right),$$

que ancora o relógio em cada retirada individual e é adequada à inspeção da latência de inferência por peça. O Apêndice B discute o *trade-off* entre A2 e A2_{lat} .

A Figura 5 apresenta o autômato temporizado correspondente à formulação A2.

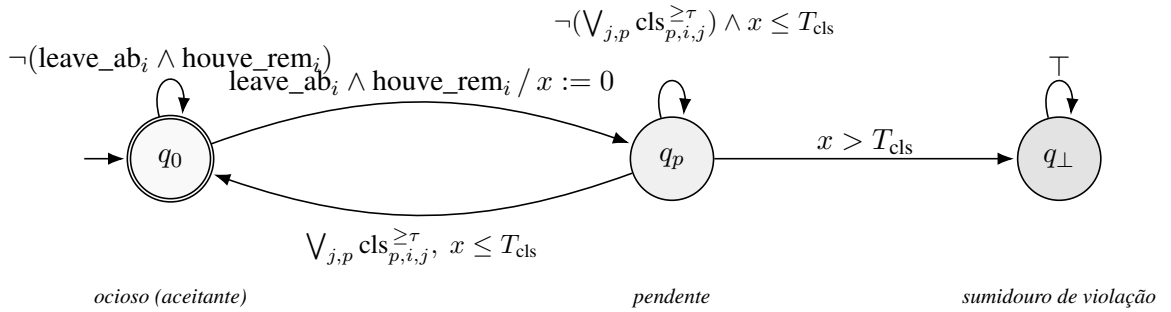


Figura 5. Autômato temporizado de monitoramento sintetizado para a propriedade A2 (*liveness* temporizada): $G((\text{leave_ab}_i \wedge \text{houve_rem}_i) \rightarrow F_{[0, T_{\text{cls}}]} \bigvee_{j \in M, p \in P} \text{cls}_{p,i,j}^{\geq \tau})$. A transição $q_0 \rightarrow q_p$ é disparada por $\text{leave_ab}_i \wedge \text{houve_rem}_i$ (encerramento de janela em que houve ao menos uma retirada) com zeragem do relógio x : a ancoragem temporal é o encerramento da janela, não a retirada individual, refletindo a *existência agregada* de classificação no intervalo $[\tau_a^i, \tau_b^i + T_{\text{cls}}]$. Em janelas inertes ($\neg \text{houve_rem}_i$) o autômato permanece em q_0 , satisfazendo A2 vacuamente. O estado q_0 é inicial e aceitante; o retorno a q_0 ocorre por alguma classificação válida sob o filtro A5 ($\bigvee_{j,p} \text{cls}_{p,i,j}^{\geq \tau}$) com guarda $x \leq T_{\text{cls}}$. A expiração do relógio sem classificação válida ($x > T_{\text{cls}}$) leva ao estado-sumidouro $q_{\perp}$, que promove o evento sintético timeout_cls_i . Os rótulos seguem a convenção “guarda / ação” dos autômatos temporizados de Alur–Dill.

A3 (*liveness* temporizada): $G(\text{leave_ab}_i \rightarrow F_{[0, T_{\text{dec}}]} (\text{match}_i \vee \text{div}_i))$

Ao encerramento da janela, o agente (via mes-bridge) deve emitir uma decisão de coerência em até T_{dec} unidades de tempo: match_i ou div_i .

Justificativa da classe. A propriedade A3 segue o padrão de *liveness* temporizada³, e não o de uma decisão *instantânea*, em razão da estrutura temporal da decisão. Uma formulação que exigisse $\text{match}_i \vee \text{div}_i$ *exatamente* no instante leave_ab_i caracterizaria uma obrigação *instantânea*, mas pressuporia a comparação $M_{\text{obs}} = M_{\text{dec}}$ atonicamente realizada em τ_b^i . Como A2' admite classificações válidas chegando até $\tau_b^i + T_{\text{cls}}$, o mes-bridge não dispõe de M_{obs} definitivamente fixado em τ_b^i . Assim, para preservar a consistência interna do conjunto de propriedades, A3 é formulada como *liveness* temporizada com prazo $T_{\text{dec}} \geq T_{\text{cls}}$. O orçamento $\epsilon := T_{\text{dec}} - T_{\text{cls}}$ acomoda o custo computacional do mes-bridge para realizar o cômputo $M_{\text{obs}}[\tau_a^i, \tau_b^i + T_{\text{cls}}] \stackrel{?}{=} M_{\text{dec}}$ e injetar o veredito no fluxo. Diferentemente de A2, A3 *não* recebe a guarda houve_rem_i : mesmo em janela inerte o mes-bridge pronuncia veredito (match_i para $M_{\text{obs}} = M_{\text{dec}} = \emptyset$), de modo que a obrigação de A3 é sempre legitimamente cumprível — sua satisfação em janela vazia é *efetiva* (por match_i), não vácuca.

Parametrização. T_{dec} é parametrizado em conjunto com T_{cls} pelo acordo de latência do mes-bridge. Para o cenário-âncora deste artigo (rotomoldagem, ciclo de aproximadamente 90 s, janela de abastecimento da ordem de 60 s), valores típicos: $T_{\text{cls}} = 1500$ ms, $\epsilon = 200$ ms, $T_{\text{dec}} = 1700$ ms.

Implicações para o diagnóstico em camadas. A classificação de A3 como *liveness* temporizada é o que permite detectar o cenário “mes-bridge mudo”: se nenhum dos vereditos match_i ou div_i é injetado até a expiração de T_{dec} , o sub-monitor M_3 alcança seu sumidouro e o efetor promove o evento sintético leave_ab_silent_i , que transita o apontamento para erro_decisao (escala à TI). A Figura 10 contrasta esse caso com a violação de A2 (falha de classificação, $\text{erro_classificacao}$), evidenciando como a composição localiza a falha na camada arquitetural correta.

A Figura 6 apresenta o autômato temporizado correspondente.

³O rótulo *liveness* temporizada designa o *padrão de especificação* (“algo bom deve ocorrer em até T unidades de tempo”); a *linguagem* efetivamente monitorada, por ter prazo finito e sumidouro absorvente, pertence à classe *safety* — toda violação tem prefixo finito irreparável, o instante $x > T$ —, conforme §5.1 e o Apêndice C.

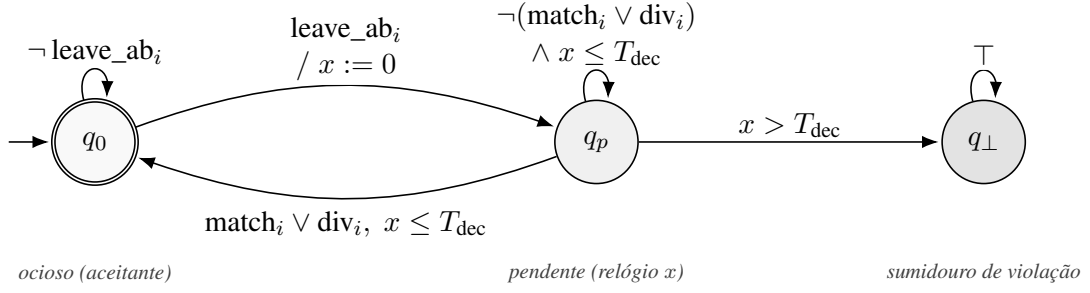


Figura 6. Autômato temporizado de monitoramento sintetizado para a propriedade A3: $G(\text{leave_ab}_i \rightarrow F_{[0, T_{\text{dec}}]}(\text{match}_i \vee \text{div}_i))$, *liveness* temporizada com prazo $T_{\text{dec}} \geq T_{\text{cls}}$, no qual o orçamento $T_{\text{dec}} - T_{\text{cls}}$ acomoda a latência do mes-bridge no cômputo da comparação multiconjunto. A figura instancia o *template* de *bounded liveness* (cf. Figura 7) com a tripla $(\text{leave_ab}_i, \text{match}_i \vee \text{div}_i, T_{\text{dec}})$. A expiração do relógio sem pronunciamento ($x > T_{\text{dec}}$) leva ao sumidouro $q_{\perp}$, que promove o evento sintético leave_ab_silent_i , consumido pelo efetor para transitar o apontamento ao status `erro_decisao` (escala à TI).

Consequência operacional. A classificação de A3 como *liveness* temporizada e a derivação de div_i fazem com que o veredito do gate de integração MES $\leftrightarrow$ ERP seja emitido em até T_{dec} após leave_ab_i , não em leave_ab_i exato. Esta latência é, no cenário-âncora, inferior a dois segundos ($T_{\text{dec}} = 1700 \text{ ms}$) — desprezível frente ao ciclo da máquina ($\sim 90 \text{ s}$) — e compatível com a janela de processamento da integração nativa MES $\rightarrow$ ERP, conforme discutido em §5.4.

A5 (restrição estrutural):⁴ apenas proposições $\text{cls}_{p,i,j}$ geradas com confiança $\geq \tau$ contribuem para M_{obs} . A5 não é uma fórmula LTL/TLTL separada, mas filtro semântico sobre o que conta como classificação válida. Arquiteturalmente, A5 atua em dois pontos de uso distintos: (i) como guarda nas transições do autômato de A2 — $\text{cls}_{p,i,j}$ com confiança inferior a τ não resolve a obrigação pendente após $\text{rem}_{i,j}$; e (ii) como pré-filtro do mes-bridge sobre o acumulador de M_{obs} (Figura 2). Esta separação evita inflar o espaço de estados do monitor: o limiar de confiança é aplicado a montante, no *pipeline* de inferência do próprio agente, em vez de ser codificado como proposição atômica adicional. A5 atua, ainda, em uma terceira frente: o filtro de confiança opera também na contagem agregada de M_{obs} ao longo de toda a janela acrescida do orçamento T_{cls} , posto que M_2 verifica existência agregada de classificação válida no intervalo.

A Tabela 2 consolida as quatro propriedades, suas classes e condições de violação.

⁴A numeração salta A4, reservada à propriedade de escalonamento ao PCP, tratada como extensão prospectiva em §6.

Tabela 2. Resumo das quatro propriedades (A1–A3 e A5) monitoradas pelo agente de visão. Classes: *safety* (algo ruim nunca acontece), *liveness temporizada* (algo bom acontece dentro de prazo), *restrição estrutural* (filtro semântico sobre o pipeline de inferência). Parâmetros: T_{cls} é a latência máxima admitida para classificação; $T_{dec} = T_{cls} + \epsilon$ é o prazo de decisão do mes-bridge, com $T_{dec} \geq T_{cls}$; τ é o limiar de confiança da inferência. A coluna Status MES indica o status terminal do apontamento disparado pelo efector quando a propriedade é violada.

ID	Classe	Fórmula LTL/TLTL	Parâm.	Semântica operacional	Condição de violação	Status MES
A1	safety	$G(rem_{i,j} \rightarrow ab_i)$	—	Toda retirada de peça ocorre dentro da janela de abastecimento.	$rem_{i,j} \wedge \neg ab_i$ (dead-end)	divergencia_pcp [‡]
A2	liveness temp.	$G((leave_ab_i \wedge houve_rem_i) \rightarrow F_{[0, T_{cls}]} \bigvee_{j \in M, p \in P} cls_{p,i,j}^{\geq \tau})$	T_{cls}	Em janela com retirada, existência de classificação válida acumulada em $[\tau_a, \tau_b + T_{cls}]$.	Expiração de T_{cls} após $leave_ab_i$ (com $houve_rem_i$) sem nenhuma $cls_{p,i,j}^{\geq \tau}$	erro_classificacao
A3	liveness temp.	$G(leave_ab_i \rightarrow F_{[0, T_{dec}]} (match_i \vee div_i))$	T_{dec}	Em até T_{dec} após $leave_ab_i$, o mes-bridge emite <i>match</i> ou <i>div</i> .	Expiração de T_{dec} sem $match_i \vee div_i$	erro_decisao
A5	restrição estrutural	filtro semântico sobre $cls_{p,i,j}$	τ	Apenas $cls_{p,i,j}$ com confiança $\geq \tau$ contribui para M_{obs} .	Não constitui fórmula LTL/TLTL (filtro a montante).	—

$div_i := mismatch_i \vee fora_ciclo_i$, macro-evento derivado materializado pelo mes-bridge; ver §3.4.

Liveness temporizada nomeia o *padrão de especificação* (“algo bom em até T ”); a linguagem efetivamente monitorada de A2/A3, por ter prazo finito e sumidouro absorvente, é da classe *safety* (toda violação tem prefixo finito ruim, a expiração de T), conforme §5.1.

[‡]A violação de A1 (retirada fora da janela) é capturada como exceção estrutural do ciclo (*fora_ciclo_i*, §3.4) e roteada a *divergencia_pcp*: o apontamento está malformado e cabe ao PCP investigá-lo. *houve_rem_i* é a guarda de não-vacuidade de A2 (verdadeira se ocorreu retirada na janela).

5. Síntese e Composição de Autômatos de Monitoramento

5.1. Síntese individual via Spot

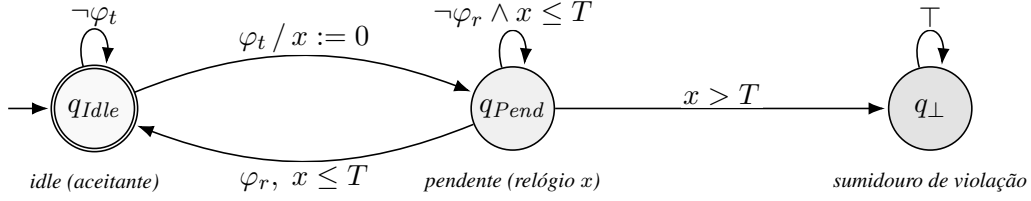
Cada propriedade A1–A3 é traduzida em autômato finito de monitoramento pela ferramenta Spot [Duret-Lutz et al. 2016], que implementa a tradução $LTL/TLTL \rightarrow \omega$ -autômatos descrita por Vardi e Wolper [Vardi and Wolper 1986] e a semântica de monitores de Bauer et al. [Bauer et al. 2011]. O Spot instancia, em particular, os algoritmos de tradução de fórmulas LTL para autômatos generalizados de Büchi e variantes [Büchi 1962, Choueka 1974], oferecendo suporte maduro à manipulação simbólica e estrutural de fórmulas e autômatos sobre palavras infinitas. Para as fórmulas deste artigo — *safety* (A1) e *bounded liveness* com prazo finito (A2', A3') —, contudo, o autômato resultante é *weak/safety* (uma única condição de aceitação), de modo que a degeneralização de Büchi generalizado é trivial e não desempenha papel na correção do produto sincronizado (§5.3). Os comandos concretos de invocação da ferramenta e exemplos de traço são apresentados no Apêndice A.

Para A1, fórmula de *safety*, o resultado é um autômato de 2 estados: um estado aceitante q_0 com auto-laço sobre o conjunto de letras que preservam a invariante, e um sumidouro de violação $q_\perp$ alcançado por toda transição não coberta. Para A2, fórmula que segue o padrão de *liveness* temporizada, o resultado é um autômato de 3 estados — um estado “ocioso”, um estado “pendente” e um sumidouro de violação —, em que a transição ocioso $\rightarrow$ pendente é disparada por $leave_ab_i \wedge houve_rem_i$, e a transição pendente $\rightarrow$ ocioso por alguma classificação válida $cls_{p,i,j}^{\geq \tau}$ dentro do prazo. A expiração do relógio sem retorno ao estado ocioso constitui violação. A síntese via Spot produz o esqueleto

qualitativo (não-temporizado) desses autômatos — artefato *intermediário* de síntese —; cada operador limitado $F_{[0,T]}$ é realizado pelo acréscimo de um único relógio que mede o tempo decorrido desde o gatilho, com a guarda $x \leq T$ e o *reset* $x := 0$ exibidos nas figuras correspondentes. Como cada propriedade temporizada emprega um único prazo, esse relógio é suficiente e a construção geral de regiões de Alur–Dill não é requerida neste recorte. A fidelidade dessa codificação repousa em dois pilares: a corretude da tradução de LTL para ω -autômato realizada pelo Spot [Duret-Lutz et al. 2016] (esqueleto qualitativo) e a semântica de autômatos temporizados de Alur–Dill [Alur and Dill 1994] para o relógio que realiza o prazo $F_{[0,T]}$; ademais, a ancoragem das obrigações em leave_ab_i assegura no máximo uma janela pendente por ciclo de braço (§3.4), de modo que um único relógio rastreia fielmente o prazo, sem acompanhar janelas sobrepostas. Em termos de aceitação ω , as Proposições 1 e 2 são enunciadas sobre o *autômato temporizado efetivamente monitorado* — com guarda $x \leq T$ e sumidouro absorvente alcançado por $x > T$ —, e não sobre o esqueleto não-temporizado. Esse autômato é um ω -autômato de *safety* (*weak*) determinístico e completo: o acréscimo do relógio e do sumidouro $q_\perp$ por expiração de prazo é precisamente o que torna a linguagem monitorada *safety* — toda violação possui um prefixo finito irreparável (o instante $x > T$ sem resolução), realizando o colapso $? \rightarrow \perp$ de [Bauer et al. 2011] no instante terminal —, e não mera restrição de monitorabilidade sobre uma propriedade de *liveness*. A aceitação de cada componente reduz-se, portanto, a *nunca atingir o sumidouro absorvente*; como $q_\perp$ é absorvente e todos os demais estados são aceitantes, a condição de Büchi coincide com essa condição de *safety/weak* (coincidência Büchi–*weak*/co-Büchi–*singleton*) [Alpern and Schneider 1985, Manna and Pnueli 1992].

Para A3, a estrutura é idêntica à de A2 — mesmo autômato de Büchi de três estados, diferindo apenas na tripla (gatilho, resolução, prazo) —, conforme o padrão de *bounded liveness* formalizado a seguir.

Padrão arquitetural de *bounded liveness*: A2 como instância canônica. A inspeção dos autômatos sintetizados revela um *template* de *bounded liveness* com três estados ($q_{\text{Idle}}, q_{\text{Pend}}, q_\perp$) parametrizados pela tripla $(\varphi_t, \varphi_r, T)$ — gatilho, resolução e prazo. A2' é a instância canônica, com a tripla $(\text{leave_ab}_i \wedge \text{houve_rem}_i, \bigvee_{j \in M, p \in P} \text{cls}_{p,i,j}^{\geq \tau}, T_{\text{cls}})$; A3' instancia o mesmo padrão com $(\text{leave_ab}_i, \text{match}_i \vee \text{div}_i, T_{\text{dec}})$. A Figura 7 apresenta o *template* e suas instanciações. A regularidade do padrão tem valor metodológico: novas propriedades operacionais que compartilhem a estrutura “algo bom deve acontecer em até T unidades de tempo após um gatilho” podem ser adicionadas ao monitor composto pela simples especialização das transições, preservando a estrutura demonstrada nas Proposições da §5.3. A propriedade A6 (*heartbeat*, §6) e as demais extensões prospectivas discutidas na mesma seção são instâncias diretas desse *template*.



Prop.	φ_t (gatilho)	φ_r (resolução)	T
A2'	$\text{leave_ab}_i \wedge \text{houve_rem}_i$	$\bigvee_{j \in M, p \in P} \text{cls}_{p,i,j}^{\geq \tau}$	T_{cls}
A3'	leave_ab_i	$\text{match}_i \vee \text{div}_i$	T_{dec}
A6	heartbeat_i	heartbeat_i	T_h
A4	div_i	esc_pcp_i	T_{pcp}

Figura 7. Template arquitetural do padrão *bounded liveness* compartilhado. As propriedades monitoradas A2' e A3' (acima da linha) instanciam o mesmo esquema de três estados: q_{Idle} (inicial e aceitante), q_{Pend} (pendente, com relógio x) e $q_{\perp}$ (sumidouro de violação). A ocorrência de φ_t dispara a transição $q_{\text{Idle}} \rightarrow q_{\text{Pend}}$ com reinicialização do relógio; a ocorrência de φ_r dentro do prazo T retorna ao estado *idle*; e a expiração do relógio sem φ_r leva ao sumidouro. A regularidade do padrão é reutilizável: as instâncias prospectivas (abaixo da linha) A6 (*heartbeat*, na variante *Armed*) e A4 (escalonamento ao PCP), discutidas na §6, especializam o mesmo *template* sem alterar a estrutura de prova composicional.

5.2. Composição por produto sincronizado

Sejam M_1, M_2, M_3 os monitores sintetizados para A1, A2', A3' respectivamente, com $M_k = (Q_k, \Sigma, \delta_k, q_{0k}, F_k)$. Definimos o monitor composto $M = M_1 \otimes M_2 \otimes M_3$ como produto sincronizado sobre o alfabeto comum $\Sigma = 2^{AP}$. Formalmente:

$$M = (Q_1 \times Q_2 \times Q_3, \Sigma, \delta, (q_{01}, q_{02}, q_{03}), F),$$

em que $\delta((s_1, s_2, s_3), a) = (\delta_1(s_1, a), \delta_2(s_2, a), \delta_3(s_3, a))$ e $F = F_1 \times F_2 \times F_3$. Cada M_k consome eventos diretamente do agente de visão e do mes-bridge — incluindo o macro-evento derivado div_i , materializado pelo mes-bridge e consumido por M_3 . Uma violação composta ocorre quando **pelo menos um** dos componentes alcança seu estado-sumidouro de violação.

Promoção de sumidouro a evento. Independentemente do produto sincronizado, o efector (*gate*) promove os sumidouros de violação de M_2 e M_3 a eventos sintéticos ($\text{timeout_cls}_i, \text{leave_ab_silent}_i$) no fluxo enriquecido $\hat{\Sigma} = \Sigma \cup \{\text{timeout_cls}_i, \text{leave_ab_silent}_i\}$, usados exclusivamente para o roteamento de status (§5.4). Esse enriquecimento é uma transdução de estado finito determinística e causal (Apêndice C, Lema 1) e, portanto, não altera a ω -regularidade nem a corretude do produto $M_1 \otimes M_2 \otimes M_3$: a promoção é função apenas do estado corrente de cada sub-monitor, sem realimentação sobre a função de transição δ .

5.3. Proposição de corretude composicional

Proposição 1 (Corretude composicional *offline*). *Seja $\sigma \in \Sigma^\omega$ um traço gerado pelo agente de visão e pelo mes-bridge, incluindo o macro-evento derivado div_i conforme §3.4. Então:*

$$\mathcal{L}_\omega(M) = \bigcap_{k=1}^3 \pi_{\Sigma_k}^{-1}(\mathcal{L}_\omega(M_k)),$$

em que Σ_k é o alfabeto sobre o qual M_k está definido e $\pi_{\Sigma_k}^{-1}$ é a operação de cilindrficação canônica para alfabeto comum Σ . Como cada M_k é um ω -autômato de safety (weak) determinístico cuja aceitação é evitar o sumidouro absorvente $\perp_k$ (i.e. $F_k = Q_k \setminus \{\perp_k\}$), o produto sincronizado M é igualmente de safety: sua aceitação é evitar o sumidouro composto, e $F = F_1 \times F_2 \times F_3$ é exatamente o conjunto de tuplas sem coordenada em sumidouro. A interseção de linguagens de safety é, então, o produto-padrão de autômatos [Baier and Katoen 2008], e a classe safety é fechada sob interseção finita [Alpern and Schneider 1985, Manna and Pnueli 1992]; nenhuma degeneralização de Büchi generalizado é requerida, pois não há condição de recorrência a sincronizar.

Proposição 2 (Corretude composicional *online*, LTL_3). *Para todo prefixo finito $\sigma^n \in \Sigma^*$, o veredito do monitor composto satisfaz:*

$$[\sigma^n \models M] = \bigcap_{k=1}^3 [\pi_{\Sigma_k}(\sigma^n) \models M_k],$$

onde $\sqcap$ denota o ínfimo na ordem $\perp < ? < \top$ (cf. [Bauer et al. 2011], Def. 4), e $\pi_{\Sigma_k}(\sigma^n)$ é a projeção do prefixo enriquecido sobre o alfabeto de M_k . A monotonicidade da operação $\sqcap$ se mantém: se algum sub-monitor alcança $\perp$, o veredito composto colapsa para $\perp$ e ali permanece; se todos os sub-monitores estão em $\top$, o composto também está em $\top$; nos demais casos, $?$. Como $A1$, $A2'$ e $A3'$ são propriedades de safety não-co-safety, o veredito $\top$ não é alcançável sobre prefixo finito — todo prefixo admite prolongamento violador —, de modo que o domínio efetivo de cada sub-monitor no stream é $\{\perp, ?\}$; a cláusula sobre $\top$ permanece vacuamente válida e não afeta a solidez nem o caso $?$.

Esboço de prova. A decomposição da prova segue dois casos:

(i) **Componentes de safety (A1, A2', A3') sobre prefixos finitos.** Os três componentes são ω -autômatos de safety (weak) determinísticos: A1 por construção, e A2'/A3' porque a *bounded liveness* $G(\varphi_t \rightarrow F_{[0,T]}\varphi_r)$ com prazo finito é da classe *safety* — toda violação possui prefixo finito irreparável, o instante $x > T$ sem resolução, conduzindo ao sumidouro absorvente [Alpern and Schneider 1985, Manna and Pnueli 1992]. Em todos a aceitação é *nunca atingir o sumidouro*, e a interseção das linguagens cilindradas é o produto finito sincronizado de autômatos de safety [Baier and Katoen 2008, Havelund and Roşu 2004], projetado sobre Σ por cilindrficação. O macro-evento derivado div_i pertence a AP e é consumido por M_3 como qualquer outra letra; sua materialização determinística pelo mes-bridge (a partir de mismatch_i e fora_ciclo_i) é causal e não introduz dependência sobre letras futuras.

(ii) **Semântica online (LTL_3).** A construção do ínfimo $\sqcap$ sobre os prefixos projetados $\pi_{\Sigma_k}(\sigma^n)$ é monotônica e preserva a estrutura de ordem $\perp < ? < \top$ no produto de

três componentes. O veredito terminal corresponde à projeção da função de transição no instante final, colapsando estados pendentes não-resolvidos de M_2 ou M_3 para $\perp$.

A demonstração completa — com o Lema da transdução determinística e as provas integrais das duas proposições — encontra-se no Apêndice C. $\square$

A Figura 8 ilustra graficamente a construção, exibindo o estado inicial s_0 , dois estados operacionais representativos e os sumidouros de violação *safety* (V_{A1}) e *liveness* temporizada (V_{A2}, V_{A3}).

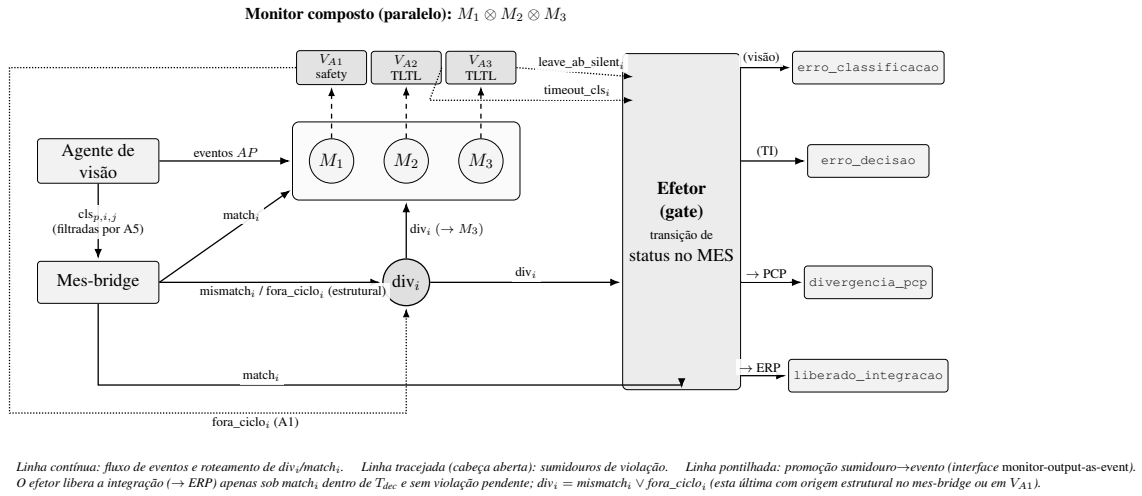


Figura 8. Monitor composto $M = M_1 \otimes M_2 \otimes M_3$ (produto sincronizado paralelo) que consome eventos do agente de visão e do mes-bridge. O agente alimenta o mes-bridge com as classificações $cls_{p,i,j}$ (filtradas pelo limiar de confiança de A5) e o monitor com os eventos de AP. Os sumidouros de violação V_{A1} (*safety*), V_{A2} e V_{A3} (*liveness* temporizada) são promovidos a eventos sintéticos consumidos pelo efetor (gate), que os roteia aos status terminais do MES: o sumidouro de M_2 ($timeout_cls_i$) leva a *erro_classificacao* (time de visão); o de M_3 ($leave_ab_silent_i$) a *erro_decisao* (TI). O macro-evento derivado $div_i = mismatch_i \vee fora_ciclo_i$ é a junção única para a qual convergem tanto as exceções fabricadas pelo mes-bridge (divergência de multiconjunto e exceções estruturais) quanto a promoção do sumidouro de A1 ($fora_ciclo_i$ por violação de *safety*); div_i é consumido por M_3 e roteado pelo efetor a *divergencia_pcp* (PCP). Por fim, $match_i$ — emitido pelo mes-bridge dentro de T_{dec} e na ausência de violação pendente — é roteado pelo efetor a *liberado_integracao*, liberando a integração nativa MES $\rightarrow$ ERP. A interface *monitor-output-as-event* preserva a localização auditável da falha.

A proposição garante que o monitor composto não introduz divergência (falsos positivos ou falsos negativos) *em relação à conjunção das especificações* — garantia relativa à especificação, não à realidade física observada pela CNN (cf. §6). Operacionalmente, isso significa que a decisão do monitor sobre liberar ou bloquear a integração MES $\rightarrow$ ERP corresponde exatamente à conjunção das decisões individuais, sem necessidade de raciocínio adicional sobre interações entre as propriedades. O Apêndice A ilustra a aplicação da composição a traços concretos, exibindo um caso de aceitação e dois casos de violação.

Verificação empírica complementar. A Proposição 2 foi adicionalmente verificada de forma empírica por *property-based testing* sobre 400 traços aleatórios (200 testes da semântica de *stream* e 200 da semântica terminal) gerados no emulador de referência associado a este trabalho. Em nenhum dos casos a invariante $\prod_k [\sigma^n \models M_k]$ foi violada pelo veredito composto. Esta evidência empírica não substitui a prova esboçada acima — nem o pretende —, mas constitui argumento metodológico complementar ao reduzir a probabilidade de erros de implementação que poderiam comprometer a fidelidade do monitor à especificação formal.⁵

5.4. Gate de decisão

Operacionalmente, a corretude composicional do monitor materializa-se em um artefato arquitetural concreto: o *gate* $MES \leftrightarrow ERP$, **realizado como a lógica de transição de status do apontamento de produção dentro do MES**. A cada nova letra e_t emitida pelo agente de visão ou pelo mes-bridge no fluxo enriquecido $\hat{\Sigma}$, o gate executa as seguintes operações em sequência: (i) atualiza o estado corrente s do monitor composto pela aplicação de δ sobre Σ ; (ii) se M_2 alcança seu sumidouro de violação neste passo, o gate consome o evento promovido $timeout_cls_i$ e transita o apontamento para *erro_classificacao*, escalando-o ao time de visão; (iii) se M_3 alcança seu sumidouro, o gate consome o evento promovido $leave_ab_silent_i$ e transita o apontamento para *erro_decisao*, escalando-o à área de TI; (iv) se a comparação multiconjunto, uma exceção estrutural ou o sumidouro de M_1 (violação de A1, diagnosticada como *safety_A1* e capturada como *fora_ciclo_i*) faz div_i se materializar, o gate transita o apontamento para *divergencia_pcp* — escalada determinística ao PCP —, anexando diagnóstico que identifica *qual fonte* disparou a divergência (*mismatch_i* ou *fora_ciclo_i*); e (v) ao observar $match_i$ no prazo T_{dec} após $leave_ab_i$ sem violação pendente, transita o apontamento para *liberado_integracao*. Esta identificação de fonte e de modo de falha é o atributo auditável central da arquitetura.

Separação de modos de falha. A arquitetura distingue formalmente quatro destinos terminais para o ciclo de vida do apontamento, roteados pelo efetor a partir do componente que detecta a falha. (1) *liberado_integracao*: todos os monitores estão satisfeitos e há $match_i$ dentro de T_{dec} ; a integração nativa $MES \rightarrow ERP$ prossegue (caminho feliz). (2) *divergencia_pcp*: materializou-se div_i (divergência de multiconjunto ou exceção estrutural); a causa-raiz é de *processo* e a escalada ao PCP é ação determinística do gate. (3) *erro_classificacao*: M_2 alcançou seu sumidouro (A2 violada, classificação ausente em T_{cls}); a causa-raiz é o agente de visão e a tratativa é encaminhada ao time de visão. (4) *erro_decisao*: M_3 alcançou seu sumidouro (A3 violada, mes-bridge mudo em T_{dec}); a causa-raiz é a camada de *integração* e a tratativa é encaminhada à área de TI. Esta política de roteamento por causa-raiz não altera a natureza dos monitores: M_1 – M_3 permanecem artefatos puramente declarativos, cuja única tarefa é decidir a satisfação de prefixos de traço. As regras “sumidouro de $M_2 \Rightarrow erro_classificacao$ ” e “sumidouro de $M_3 \Rightarrow erro_decisao$ ” são parte da especificação do *efetor* (o gate /

⁵O *property-based testing* foi implementado em QuickCheck sobre o emulador em Haskell. A geração aleatória cobre traços com diferentes combinações de A1–A3 satisfeitas e violadas. O código do emulador e dos testes acompanha o artefato depositado (cf. nota anterior).

mes-bridge), e não dos *monitores*: os autômatos apenas emitem o veredito $\perp$; é o efetor que o consome e o traduz em transição operacional de status.

A separação entre o estado do monitor e a ação do *gate* estabelece uma fronteira de responsabilidade explícita: o monitor é puramente declarativo — sua única tarefa é decidir se um prefixo de traço satisfaz a especificação —, enquanto o *gate* encapsula a política operacional (bloquear, liberar, escalar). Esta separação confere auditabilidade ao sistema: as decisões do *gate* são determinísticas e verificáveis a partir do traço de eventos e do estado do monitor, sem dependência da caixa-preta da rede neural de inferência. A Figura 9 apresenta o fluxograma correspondente.

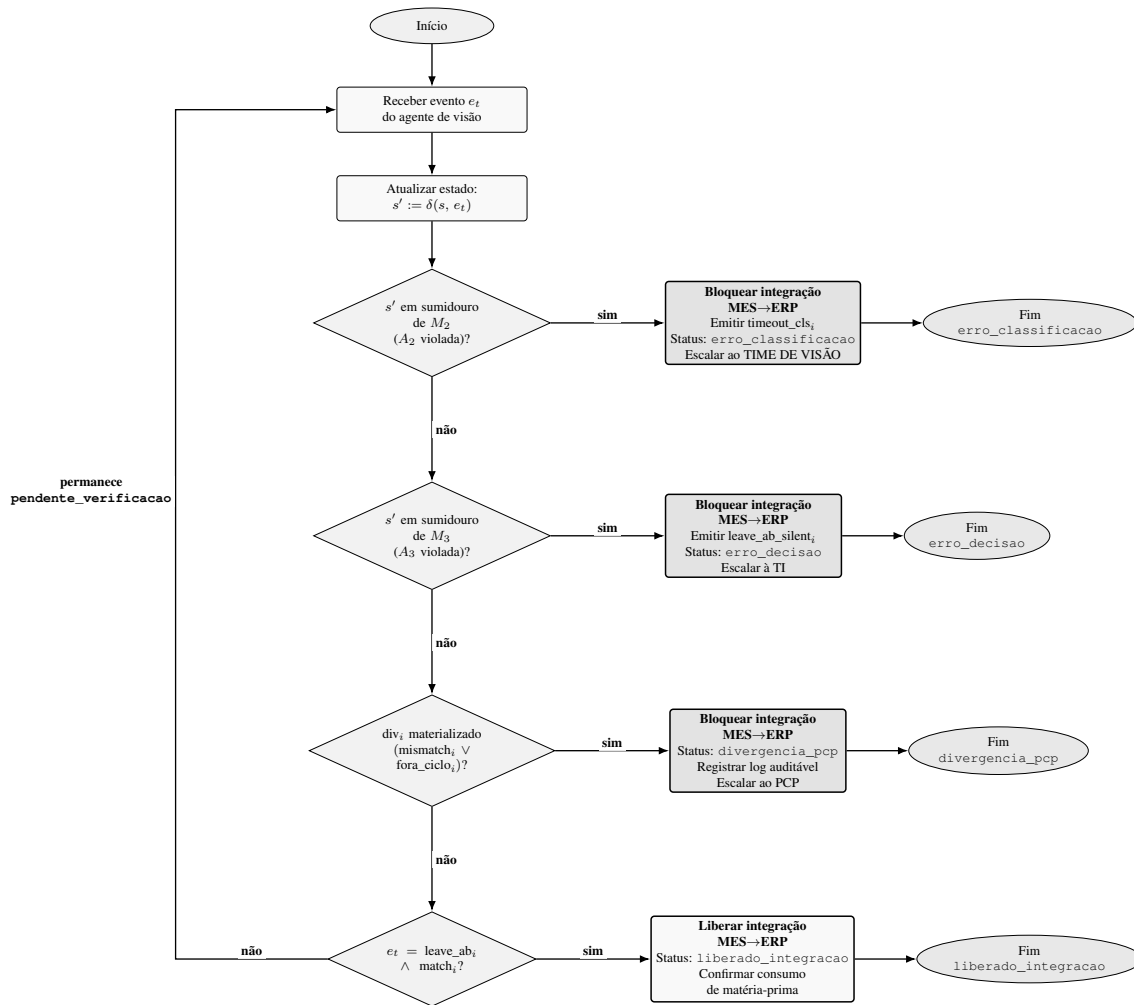


Figura 9. Fluxograma de decisão do gate de integração MES↔ERP. A cada evento e_t , o monitor composto $M = M_1 \otimes M_2 \otimes M_3$ atualiza seu estado s via δ e o gate roteia por *causa-raiz* para um de cinco desfechos. Se s' atinge o sumidouro de M_2 (A_2 violada, liveness temporal de classificação), o gate bloqueia, emite `timeout_clsi`, fixa o status `erro_classificacao` e escala ao TIME DE VISÃO. Se s' atinge o sumidouro de M_3 (A_3 violada, liveness temporal de decisão), o gate bloqueia, emite `leave_ab_silenti`, fixa `erro_decisao` e escala à TI (Tecnologia da Informação). Caso div_i se materialize (`mismatchi v fora_cicloi`), o gate bloqueia com status `divergencia_pcp` e escala ao PCP (tratativa manual determinística). Quando $e_t = leave_ab_i$ acompanhado de `matchi` (encerramento válido de janela em T_{dec}), a integração é *liberada* com status `liberado_integracao` e o consumo de matéria-prima é confirmado no ERP. Nenhum dos casos anteriores mantém a janela em `pendente_verificacao`, estado transitório que sedia o gate, e o fluxo retorna ao laço aguardando o próximo evento. Os sumidouros de M_2 e M_3 são terminais, cada um com seu status próprio.

Diagnóstico em camadas: A2 versus A3. A introdução do mes-bridge tem como consequência um *refino da camada de detecção* das falhas. Considerem-se cenários operacionais como “Ordem de Produção errada” ou “molde vazio esquecido”, em que a janela é encerrada sem que o multiconjunto observado coincida com o declarado. Sem o

mes-bridge, o agente termina a janela sem emitir match_i nem div_i , e a falha cai indistintamente sob a mesma camada operacional: o MES registra *erro_classificacao* sem que se possa distinguir se a falha foi do agente de visão (a classificação não chegou) ou da camada de decisão (o *mes-bridge* ficou mudo). Com o *mes-bridge*, A2 e A3 discriminam as causas: A2 é violada quando o agente de visão não classificou no prazo T_{cls} (*status erro_classificacao*, encaminhamento ao time de visão); A3 é violada quando a classificação completou mas o *mes-bridge* não pronunciou veredito em T_{dec} (*status erro_decisao*, encaminhamento à TI). A composição do monitor faz com que a falha aponte para a camada arquitetural correta. A Figura 10 explicita os dois cenários lado a lado, e a Tabela 3 sintetiza os cenários possíveis e o status MES resultante em cada um — evidenciando que o status é função quase exclusiva do par $(\text{match}_i, \text{div}_i)$ e dos prazos $(T_{\text{cls}}, T_{\text{dec}})$.

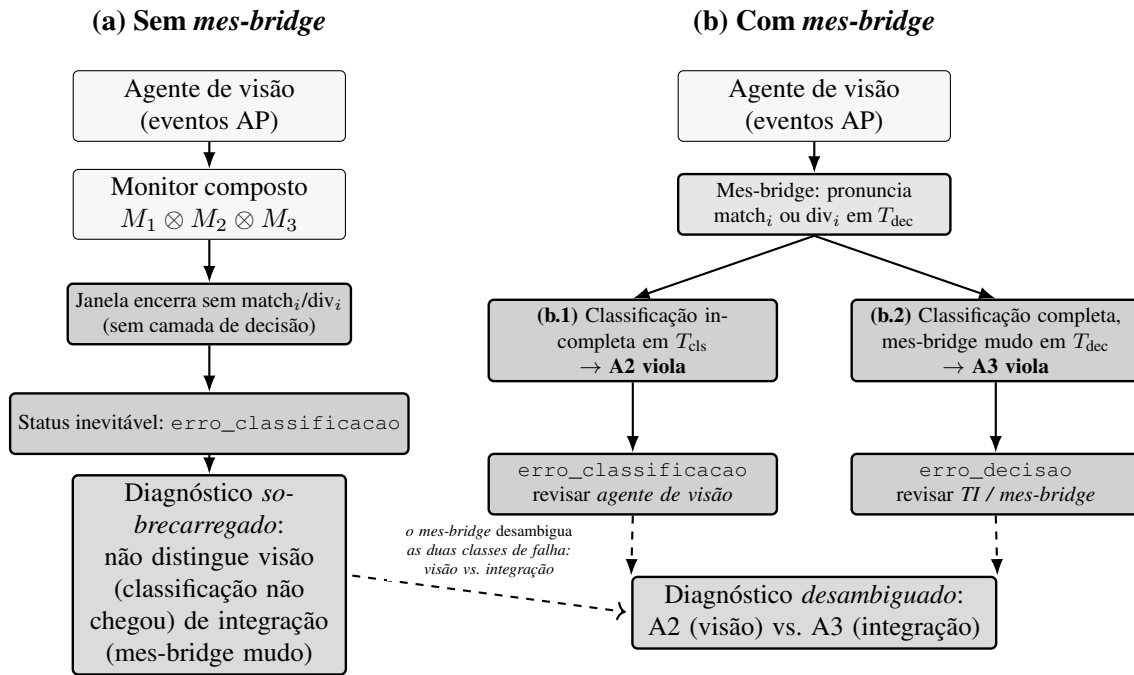


Figura 10. Diagnóstico em camadas com a introdução do *mes-bridge*. No cenário (a) — sem *mes-bridge* —, a janela de abastecimento encerra sem $\text{match}_i/\text{div}_i$ e o único status possível é *erro_classificacao*; esse status é *semanticamente sobrecarregado*, pois falhas do agente de visão (a classificação não chegou) e da camada de decisão (*mes-bridge* mudo) colapsam no mesmo rótulo. No cenário (b) — com *mes-bridge* —, as propriedades de *liveness* temporizada A2 e A3 discriminam as duas causas: em (b.1), classificação incompleta em T_{cls} faz A2 violar, roteando a *erro_classificacao* (revisar o *agente de visão*); em (b.2), classificação completa mas *mes-bridge* mudo em T_{dec} faz A3 violar, roteando a *erro_decisao* (revisar *TI / mes-bridge*). A desambiguação preserva a localização auditável da falha (transparência estrutural do monitor composto).

Tabela 3. Status do apontamento no MES em função dos sinais do agente de visão e dos marcos temporais. O status é função quase exclusiva do par $(match_i, div_i)$: quando ambos são F, os timeouts T_{cls} e T_{dec} é que determinam o veredito. `erro_classificacao` é absorvente — alcançado primeiro quando não há classificação válida; o cenário 6 pressupõe A2 satisfeita (houve classificação). $match_i$ e div_i são mutuamente exclusivos (o mes-bridge emite no máximo um). O cenário 4 desdobra-se segundo a origem de div_i : $mismatch_i$ (divergência de multiconjunto, com A1–A3 satisfeitas) ou $fora_ciclo_i$ (exceção estrutural, com A1 violada). A guarda $houve_rem_i$ de A2 (§4) assegura que janelas inertes, sem retirada, não disparam o cenário 5.

#	match	div	Marco temporal	Status MES	Prop.	Encaminhamento
1	F	F	antes de τ_b	pendente_verificacao	—	nenhum
2	F	F	em $[\tau_b, \tau_b + T_{cls}]$	pendente_verificacao	A2 pendente	nenhum
3	T	F	qualquer após τ_b	liberado_integracao	A1, A2, A3 ok	nenhum
4a	F	T	qualquer após τ_b (origem $mismatch_i$)	divergencia_pcp	A1, A2, A3 ok	PCP
4b	F	T	qualquer após τ_b (origem $fora_ciclo_i$)	divergencia_pcp	A1 violada	PCP
5	F	F	T_{cls} expirado (sem classificação válida)	erro_classificacao	A2 violada	agente de visão
6	F	F	T_{dec} expirado (houve classificação, sem veredito)	erro_decisao	A3 violada	TI / mes-bridge

A Tabela 4 consolida o mapeamento entre componentes do monitor, detecção de violação e transição MES disparada pelo *gate*.

Tabela 4. Mapeamento entre componentes do monitor composto, condição de detecção, veredito disparador e transição de status no MES. O efector (*gate*) roteia cada modo de falha a um status terminal distinto: o sumidouro de M_2 leva a `erro_classificacao` (escala ao time de visão); o de M_3 a `erro_decisao` (escala à TI); o macro-evento derivado $div_i = mismatch_i \vee fora_ciclo_i$ a `divergencia_pcp` (escala determinística ao PCP); e $match_i$ a `liberado_integracao`. As regras “sumidouro $\Rightarrow$ status” são especificação do efector *mes-bridge*, não dos autômatos, que apenas emitem o veredito $\perp$.

Componente	Condição de detecção	Veredito disparador	Transição MES
M_1 (A1, <i>safety</i>)	$rem_{i,j}$ fora da janela ab_i	sumidouro de $M_1 \rightarrow fora_ciclo_i$	$\rightarrow divergencia_pcp$
M_2 (A2', <i>liveness temp.</i>)	classificação não acumulada em T_{cls} pós $leave_{ab_i}$	sumidouro de $M_2 \rightarrow timeout_cls_i$	$\rightarrow erro_classificacao$ (efetor, escala visão)
M_3 (A3', <i>liveness temp.</i>)	<i>mes-bridge</i> silente em T_{dec}	sumidouro de $M_3 \rightarrow leave_{ab_silente_i}$	$\rightarrow erro_decisao$ (efetor, escala TI)
<i>mes-bridge</i> (mismatch)	$M_{obs} \neq M_{dec}$ em $leave_{ab_i} + \epsilon$	$mismatch_i \rightarrow div_i$	$\rightarrow divergencia_pcp$ (escala PCP)
<i>mes-bridge</i> (estrutural)	exceção estrutural do ciclo	$fora_ciclo_i \rightarrow div_i$	$\rightarrow divergencia_pcp$ (escala PCP)
caminho de aceitação	todos os monitores $\models \top$ e $match_i$ em T_{dec}	composto $\models \top$	$\rightarrow liberado_integracao$

5.5. Complexidade

Seja $\Phi = \{A1, A2, A3\}$ o conjunto de propriedades monitoradas, e $N_{SKU} = |P|$. Os monitores componentes têm: M_1 (A1, *safety*) 2 estados; M_2 (A2', *liveness temp.*) 3 estados;

M_3 (A3', liveness temp.) 3 estados. O produto sincronizado tem, portanto, no máximo $\prod_{k=1}^3 |Q_k| = 2 \times 3 \times 3 = 18$ estados, cota constante e independente do tamanho do traço observado. Alternativamente, sob a abstração binária (operacional vs. violação) por componente, a cota se reduz a $2^{|\Phi|} = 8$. A atualização do monitor por evento é $O(|\Phi|)$, pois cada componente realiza uma transição em tempo constante.

A cota binária permanece em $2^{|\Phi|} = 8$, posto que cada sub-monitor é classificado como “operacional vs. violação” para fins do gate. O custo adicional de $O(1)$ por evento para a promoção dos sumidouros de M_2 e M_3 a eventos sintéticos (enriquecimento do fluxo $\hat{\Sigma}$ para roteamento de status) é absorvido no termo $O(|\Phi|)$ original.

A memória adicional necessária para manter o multiconjunto observado M_{obs} é $O(N_{\text{SKU}})$, correspondente ao contador por SKU. A complexidade espacial total do monitoramento é, portanto, $O(N_{\text{SKU}} + |\Phi|)$, enquanto a complexidade temporal por evento é $O(|\Phi|)$. Em ambos os casos, as cotas são compatíveis com execução em dispositivo de borda industrial [Törngren et al. 2021], requisito arquitetural relevante para o cenário de aplicação.

A Tabela 5 consolida as cotas de complexidade por componente do monitor; o pseudocódigo correspondente do laço de monitoramento *online* é apresentado no Algoritmo 1. A cada evento e_t , o procedimento (i) aplica o filtro A5 sobre M_{obs} ; (ii) atualiza o estado do monitor composto $M = M_1 \otimes M_2 \otimes M_3$ via δ ; (iii) deixa o mes-bridge emitir $\text{match}_i/\text{mismatch}_i$ em $\text{leave_ab}_i + \epsilon$ e fora_ciclo_i em exceção estrutural, materializando $\text{div}_i = \text{mismatch}_i \vee \text{fora_ciclo}_i$; e (iv) roteia o status do apontamento conforme o componente que detecta a falha.

Tabela 5. Cotas de complexidade do monitoramento, por componente, em notação $O(\cdot)$. Seja $|\Phi| = 3$ o número de propriedades monitoradas e $N_{\text{SKU}} = |P|$ a cardinalidade do conjunto de SKUs admissíveis. A contagem de estados de cada M_k inclui o sumidouro de violação. A cota $\leq 2^{|\Phi|}$ para o monitor composto M assume classificação binária por componente (operacional vs. violação), conforme convenção adotada na §5.5. A cota não-binária do produto é 18 estados; a cota binária $2^{|\Phi|} = 8$ é independente da classe de cada sub-monitor.

Componente	Classe	#Estados	Tempo / evento	Espaço
M_1 (A1)	safety	2	$O(1)$	$O(1)$
M_2 (A2')	liveness temp.	3	$O(1)$	$O(1)$ (relógio x)
M_3 (A3')	liveness temp.	3	$O(1)$	$O(1)$ (relógio x)
M_{obs}	multiconjunto observado	—	$O(1)$ por $\text{cls}_{p,i,j}$	$O(N_{\text{SKU}})$
$M = \bigotimes_{k=1}^3 M_k$	monitor composto	≤ 18	$O(\Phi)$	$O(\Phi)$
Total agregado	—	—	$O(\Phi)$	$O(N_{\text{SKU}} + \Phi)$

Algoritmo 1. Laço de monitoramento *online* do *gate* MES $\leftrightarrow$ ERP, consumido evento a evento pelo *gate*. Custo $O(|\Phi|)$ por evento (Tabela 5).

Entrada: traço $\sigma = e_1 e_2 \dots \in \Sigma^\omega$; estado inicial q_0 ; função $\delta : Q \times \Sigma \rightarrow Q$; sumidouros $F_\perp \subseteq Q$; limiar $\tau \in [0, 1]$; declarado M_{dec} ; orçamentos T_{cls}, T_{dec}

Saída: LIBERAR ou BLOQUEAR(*status*) com *status* $\in \{\text{divergencia_pcp}, \text{erro_classificacao}, \text{erro_decisao}\}$ e *diag* $\in \{\text{safety_A1}, \text{mismatch}, \text{fora_ciclo}, \text{timeout_cls}, \text{leave_ab_silent}\}$

```

1:  $s \leftarrow q_0$ ;  $M_{obs} \leftarrow \emptyset$ ;  $armado \leftarrow \text{FALSO}$ ;  $t_b \leftarrow \perp$ 
2: para cada evento  $e_t \in \sigma$  no instante  $t$  faça
3:   se  $e_t = \text{cls}_{p,i} \wedge \text{conf}(e_t) \geq \tau$  então
4:      $M_{obs}[p] \leftarrow M_{obs}[p] + 1$  ▷ A5: filtro semântico
5:   fim se
6:   se  $e_t = \text{leave\_ab}_i$  então
7:      $armado \leftarrow \text{VERD}$ ;  $t_b \leftarrow t$  ▷ abre horizonte de decisão
8:   fim se
9:    $s \leftarrow \delta(s, e_t)$  ▷ atualização em  $O(|\Phi|)$  sobre  $\Sigma$ 
10:   $div \leftarrow \text{FALSO}$ 
11:  se  $armado \wedge t \geq t_b + \epsilon$  então ▷ mes-bridge compara  $M_{obs}$  e  $M_{dec}$ 
12:    se  $M_{obs} = M_{dec}$  então
13:      emitir  $match_i$ 
14:    senão
15:      emitir  $mismatch_i$ ;  $div \leftarrow \text{VERD}$ ;  $diag \leftarrow \text{mismatch}$ 
16:    fim se
17:  fim se
18:  se  $\text{excecao\_estrut}(e_t, s)$  então
19:    emitir  $\text{fora\_ciclo}_i$ ;  $div \leftarrow \text{VERD}$ ;  $diag \leftarrow \text{fora\_ciclo}$ 
20:  fim se
21:  se  $div$  então ▷ deriva  $div_i$  e resolve a obrigação de  $M_3$ 
22:     $s \leftarrow \delta(s, div_i)$ 
23:  fim se
24:  se  $\text{sink}_{M_1}(s)$  então ▷ violação safety estrutural (A1)
25:     $\text{pendente\_verificacao} \rightarrow \text{divergencia\_pcp}$ ; retornar
    BLOQUEAR( $\text{divergencia\_pcp}$ ,  $\text{safety\_A1}$ )
26:  fim se
27:  se  $\text{sink}_{M_2}(s)$  então ▷ A2 violada: classificação ausente em  $T_{cls}$ 
28:    emitir  $\text{timeout\_cls}_i$ ;  $\text{pendente\_verificacao} \rightarrow \text{erro\_classificacao}$ 
29:    retornar BLOQUEAR( $\text{erro\_classificacao}$ ,  $\text{timeout\_cls}$ )
30:  fim se
31:  se  $\text{sink}_{M_3}(s)$  então ▷ A3 violada: mes-bridge mudo em  $T_{dec}$ 
32:    emitir  $\text{leave\_ab\_silent}_i$ ;  $\text{pendente\_verificacao} \rightarrow \text{erro\_decisao}$ 
33:    retornar BLOQUEAR( $\text{erro\_decisao}$ ,  $\text{leave\_ab\_silent}$ )
34:  fim se
35:  se  $div$  então ▷  $div_i$  disparou: escala determinística ao PCP
36:     $\text{pendente\_verificacao} \rightarrow \text{divergencia\_pcp}$ ; retornar
    BLOQUEAR( $\text{divergencia\_pcp}$ ,  $diag$ )
37:  fim se
38:  se  $match_i$  emitido em  $t \leq t_b + T_{dec}$  então ▷ coerência confirmada
39:     $\text{pendente\_verificacao} \rightarrow \text{liberado\_integracao}$ ; retornar LIBERAR
40:  fim se
41: fim para

```

6. Discussão e Trabalhos Futuros

A separação entre a especificação formal (LTL/TLTL) e a implementação do agente (rede neural de visão) é deliberada e oferece três vantagens. Primeira, o monitor é auditável

independentemente do modelo de inferência: alterações no *backbone* do agente não invalidam a especificação. Segunda, o monitor é certificável: a corretude da composição (Proposições 1 e 2) é estabelecida por argumento dedutivo, complementado — não substituído — pela verificação empírica do emulador. Terceira, o monitor é portátil: a mesma especificação pode ser instanciada em outras células de produção que compartilhem a topologia de janela operacional.

As subseções seguintes registram extensões prospectivas que o arcabouço comporta — não implementadas nesta versão e situadas fora do recorte avaliado; a contribuição efetiva deste artigo limita-se às propriedades A1–A3 e A5 e à arquitetura monitor / mes-bridge / gate já formalizada.

Heartbeat com semântica Armed/Unarmed (A6). O arcabouço admite extensões naturais. Uma extensão particularmente útil é a inclusão de uma propriedade de sinal de vida (*heartbeat*) do agente, denotada A6. Em sua forma literal, $G F_{[0, T_h]} \text{heartbeat}_i$ expressa adequadamente a exigência de cadência mínima, mas é violada por todo traço histórico em que o mecanismo de *heartbeat* não estava ativo — incluindo, por construção, todos os traços operacionais anteriores à sua introdução. Adota-se aqui uma variante *opt-in*, com semântica *Armed/Unarmed*, que enfraquece a forma literal para $G(\text{heartbeat}_i \rightarrow F_{[0, T_h]} \text{heartbeat}_i)$, aplicada a partir da primeira ocorrência de *heartbeat*_{*i*}: o autômato inicia em estado *unarmed* (vacuamente satisfeito) e somente passa a exigir cadência após o primeiro *heartbeat*. Esta variante preserva a essência operacional da propriedade, tem precedente em construções “initially” de propriedades de *liveness* e é compatível com a estrutura de prova das Proposições 1 e 2, uma vez que se enquadra no padrão arquitetural compartilhado de *bounded liveness* (Figura 7). A Figura 11 apresenta o autômato correspondente.

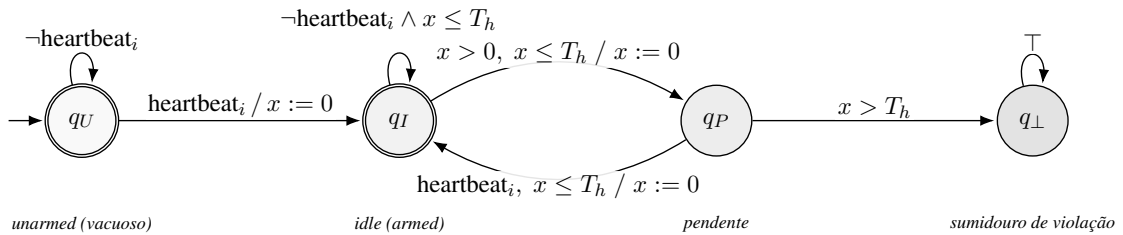


Figura 11. Autômato temporizado de monitoramento da propriedade A6 (sinal de vida do agente, *heartbeat*) com semântica *Armed/Unarmed*. Em sua forma literal, $G F_{[0, T_h]} \text{heartbeat}_i$ é violada por todo traço histórico que não contenha *heartbeat*_{*i*} — inclusive aqueles anteriores à introdução do mecanismo. A refinement *opt-in* adotada substitui A6 pela variante operacional “ $G(\text{heartbeat}_i \rightarrow F_{[0, T_h]} \text{heartbeat}_i)$ a partir da primeira ocorrência”: o autômato inicia em q_U (*unarmed*, vacuamente satisfeito) e somente “arma” em q_I após o primeiro *heartbeat*_{*i*}. A partir desse instante, a expiração de T_h sem nova ocorrência leva ao sumidouro $q_\perp$. A construção tem precedente em variantes “initially” de propriedades de *liveness* e preserva a essência operacional da exigência de cadência.

Extensão para SLA externo: A4 como exemplar de *bounded liveness*. Uma extensão natural do conjunto monitorado é uma propriedade A4 de *bounded liveness* sobre

o escalonamento ao PCP: $A4 \equiv G(\text{div}_i \rightarrow F_{[0, T_{\text{pcp}}]} \text{esc_pcp}_i)$. A4 instancia o mesmo *template* arquitetural de A2 (gatilho, resolução e prazo; Figura 7) e ilustra como o *framework* comporta SLAs externos verificáveis. A presente versão do sistema integra a escalada ao PCP como ação determinística do *gate* — consequência imediata do status *divergencia_pcp* —, de modo que A4 não tem efeito observável no status MES e foi deixada como propriedade auxiliar para auditoria fora do laço operacional. Sua inclusão em sistemas com múltiplos consumidores do veredito (*dashboard* de SLA, monitoramento contratual, integração com sistemas *downstream* sujeitos a falha) é trabalho futuro direto.

Eventos derivados como meta-padrão. A formulação de div_i como evento derivado, adotada neste trabalho, sugere um meta-padrão arquitetural reutilizável: *promoção de sub-monitor a evento sintético do fluxo*. O padrão tem três ingredientes: (a) interface “monitor-output-as-event” explícita; (b) cilindricação determinística para alfabeto estendido; (c) aciclicidade da dependência entre sub-monitores promovidos. Cumpridas estas três condições, a estrutura de prova das Proposições 1 e 2 generaliza-se diretamente. Esta abstração abre caminho para futuras propriedades de meta-monitoramento sem recurso a pontos fixos ou aproximações.

Extensão direta para múltiplos braços. O monitor composto formalizado neste artigo opera sobre os eventos de um único braço i . Em produção, a máquina de rotomoldagem possui três braços operando simultaneamente, cada um com sua própria janela de abastecimento e Ordens de Produção independentes. A extensão é direta e não exige modificação da especificação formal: basta instanciar uma cópia do monitor composto por braço, com estados independentes, organizadas em uma estrutura indexada por identificador de braço (mapa $\text{ArmId} \rightarrow \text{MonitorState}$). A corretude composicional (Proposições 1 e 2) preserva-se trivialmente sob esta replicação, pois os alfabetos por braço são disjuntos (proposições indexadas por i) e a interação entre braços ocorre apenas em camadas superiores (sincronização do ciclo da máquina, fora do escopo da janela de abastecimento).

Demais loci de verificação. O arcabouço estende-se naturalmente para outros loci de verificação no mesmo sistema, atualmente fora do escopo deste artigo. Em primeiro lugar, o ciclo completo da máquina (abastecimento $\rightarrow$ forno $\rightarrow$ resfriamento) admite especificação como autômato de estados finitos paralelo, sobre o qual propriedades de *safety* relativas à sincronização de braços podem ser enunciadas. Em segundo lugar, a classificação formal de paradas (planejadas *versus* não planejadas) e a derivação dos indicadores agregados OEE, MTBF e MTTR podem ser formuladas como propriedades adicionais sobre o mesmo alfabeto estendido. A arquitetura PROSA [Van Brussel et al. 1998] oferece referência consolidada para o desdobramento holônico dessas extensões. Estes desenvolvimentos compõem o horizonte natural da dissertação aplicada que ancora o presente trabalho, sem competir com o recorte teórico ora delimitado.

Propriedade adicional com efeito colateral (A7). Uma propriedade adicional A7, que verifica a coerência da operação de refugo — pertinente quando uma peça classificada

é posteriormente descartada por defeito visual —, exibe característica peculiar: ao ser satisfeita, induz um *efeito colateral* sobre M_{obs} , qual seja, o decremento da contagem do SKU rejeitado. A7 é, portanto, ao mesmo tempo, uma propriedade de *safety* sobre o traço e uma operação sobre o multiconjunto observado mantido pelo mes-bridge. Esta semântica não compromete a Proposição 2, pois o efeito colateral atua sobre M_{obs} a montante do monitor composto (na camada do mes-bridge), preservando a separação declarativa do monitor. A inclusão formal de A7 e a discussão de sua semântica composta ficam reservadas para um desdobramento futuro.

Limitações e robustez do veredito. Uma limitação metodológica deve ser registrada. A especificação parte da premissa de que o agente de visão é a fonte fidedigna dos sinais. A robustez do monitor diante de falhas catastróficas do agente — por exemplo, perda do *stream* de imagens ou colapso do classificador — exige instrumentação adicional, como monitor de saúde (*heartbeat*, A6) e detecção de *out-of-distribution*. Estes mecanismos podem ser incorporados como propriedades adicionais, mantendo o aparato composicional aqui estabelecido. Em particular, na ausência total de eventos do agente, A2 e A3 são vacuamente satisfeitas — o que motiva a adoção de A6 na variante *Armed/Unarmed* discutida acima.

A confiabilidade do veredito do *gate* depende criticamente da calibração do limiar τ e da matriz de confusão do classificador CNN. A Figura 12 apresenta essa matriz, no conjunto de teste, para um classificador MobileNetV3-Small treinado sobre o catálogo de SKUs do cenário-âncora: a massa concentra-se na diagonal, mas confusões residuais entre SKUs visualmente próximos (p.ex. *caixa_500L* e *caixa_1000L*) persistem. Falsos negativos sob baixa luminosidade ou oclusão fazem $|M_{\text{obs}}| < |M_{\text{dec}}|$, disparando div_i mesmo quando a produção real é correta; o filtro A5 (limiar τ) atenua a montante as classificações de baixa confiança, mas não elimina o risco. Cabe sublinhar que essa matriz caracteriza o *componente de inferência* (tratado como caixa-preta pelo monitor), e não a operação do *gate* sobre traços de campo, que permanece trabalho futuro. Sugere-se calibrar τ por percentis empíricos sobre *runs* históricos e, em desenvolvimento futuro, estender o arcabouço para *statistical model checking* [Younes and Simmons 2002] ou *shielding* probabilístico [Junges et al. 2021], abordagens complementares quando o componente de inferência tem caráter intrinsecamente probabilístico.

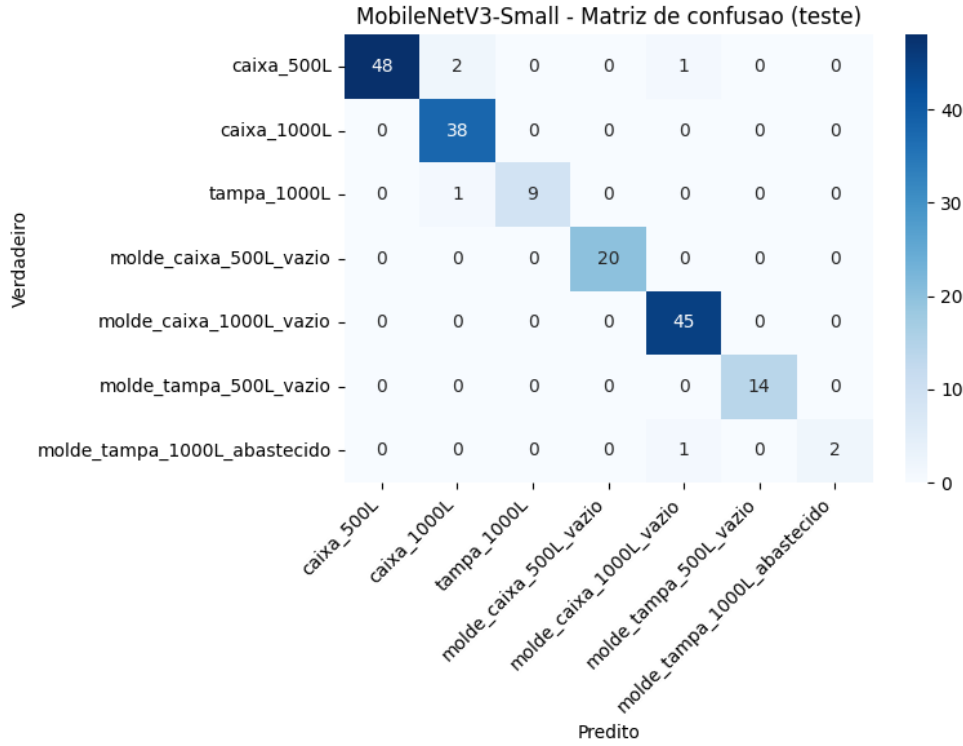


Figura 12. Matriz de confusão, no conjunto de teste, de um classificador MobileNetV3-Small treinado sobre o catálogo de SKUs do cenário-âncora (caixas e tampas de 500/1000 L e os respectivos moldes vazios/abastecido). A massa concentra-se na diagonal (alta acurácia), mas persistem confusões residuais entre SKUs visualmente próximos — p.ex. `caixa_500L` classificada como `caixa_1000L`. São exatamente essas confusões e os falsos negativos sob oclusão ou baixa luminosidade que (i) o filtro A5 (limiar τ , §4) atenua a montante e (ii) podem, quando não filtrados, materializar div_i por $|M_{obs}| \neq |M_{dec}|$. A figura caracteriza o *componente de inferência* (tratado como caixa-preta pelo monitor), não a operação do *gate* sobre traços de campo — esta última permanece trabalho futuro (§6).

Fortalecimento da confiança das classificações. O fortalecimento do grau de confiança das classificações que alimentam M_{obs} pode ser perseguido em duas frentes complementares, ambas a montante do monitor e, portanto, sem inflar seu espaço de estados. A primeira é a *calibração* das saídas do classificador CNN: redes profundas são tipicamente mal-calibradas, no sentido de que o escore de confiança $\text{conf}(\text{cls}_{p,i,j})$ reportado não corresponde à probabilidade empírica de acerto [Guo et al. 2017]. Técnicas como *temperature scaling* ou regressão isotônica reajustam esses escores de modo que a comparação $\text{conf}(\text{cls}_{p,i,j}) \geq \tau$ passe a ter significado probabilístico estável — condição necessária para que τ seja um corte interpretável, e não arbitrário. A segunda frente substitui o limiar empírico por uma garantia *distribution-free*: sob *conformal prediction* [Vovk et al. 2005, Angelopoulos and Bates 2023], τ é derivado de uma taxa-alvo de erro α a partir de um conjunto de calibração, assegurando cobertura $1 - \alpha$ sob a hipótese de permutabilidade (*exchangeability*) entre calibração e operação. Essa hipótese é tensionada justamente pelo regime de *drift* do classificador discutido nesta seção, motivando variantes adaptativas [Gibbs and Candès 2021]. Ainda assim, a abordagem converte o filtro A5

de heurística em filtro com garantia estatística explícita, no mesmo espírito de incorporar garantias estatísticas ao arcabouço — ao lado da extensão para *statistical model checking* [Younes and Simmons 2002]. Resta o compromisso fundamental entre confiança e cobertura: valores elevados de τ reduzem contagens espúrias em M_{obs} , mas, ao descartar classificações legítimas de baixa confiança, produzem dois efeitos operacionais distintos — (i) obrigações de A2 deixadas pendentes, roteadas pelo efector a `erro_classificacao` (escala à visão); e (ii) subcontagem $|M_{\text{obs}}| < |M_{\text{dec}}|$ que materializa div_i espúrio e escala a `divergencia_pcp` (PCP). Note-se que A3 permanece invariante a τ — o mes-bridge pronuncia match_i ou div_i dentro de T_{dec} em qualquer caso —, de modo que o limiar afeta o conteúdo do veredito, não a *liveness* que A3 monitora. Essa curva pode ser quantificada empiricamente sobre *runs* históricos, e seu ponto de operação deve ser fixado pelo acordo de nível de serviço entre operação, visão e PCP.

Confiabilidade do efector mes-bridge. A política de transição de status assume durabilidade e atomicidade da gravação do estado terminal — em particular, das transições para `erro_classificacao` e `erro_decisao` quando M_2 ou M_3 emite $\perp$. Falhas catastróficas do próprio *mes-bridge*, como a queda de processo entre a emissão do veredito $\perp$ por um sub-monitor e a persistência do novo status no MES, envolvem técnicas clássicas de tolerância a falhas em sistemas distribuídos — *checkpointing*, transições idempotentes, *outbox pattern* — e ficam fora do escopo deste trabalho, que recorta o problema à camada de *runtime verification*.

7. Conclusão

Este artigo apresentou a especificação formal de quatro restrições operacionais — três em LTL/TLTL e uma estrutural (A5) — que um agente industrial de visão computacional deve respeitar ao validar o apontamento de produção, e demonstrou a síntese dos autômatos finitos de monitoramento correspondentes, **incluindo a promoção dos sumidouros de M_2 e M_3 a eventos sintéticos para o roteamento auditável de status**. A composição dos monitores individuais por produto sincronizado produz um monitor agregado cuja corretude foi enunciada sob duas semânticas complementares — ω -regular (Proposição 1) e LTL₃ de três valores (Proposição 2) —, ambas com esboço de prova; a segunda foi adicionalmente verificada empiricamente por *property-based testing* sobre 400 traços aleatórios. A complexidade do monitoramento é $O(|\Phi|)$ por evento e $O(N_{\text{SKU}} + |\Phi|)$ em espaço — cotas compatíveis com execução em borda industrial.

As contribuições do trabalho se organizam em três planos. Em plano **teórico**, a delimitação das quatro propriedades operacionais e a demonstração de que a composição preserva a corretude, agora apresentada sob ambas as semânticas. Em plano **arquitetural**, a proposição de três componentes formalmente especificados — monitor composto, *mes-bridge* e *gate* $\text{MES} \leftrightarrow \text{ERP}$ —, deslocando a operação de um regime reativo, em que erros são corrigidos após se manifestarem no ERP, para um regime preventivo ancorado em verificação formal em tempo real, com diagnóstico em camadas (§5.4) e separação de modos de falha entre o PCP (divergência operacional, `divergencia_pcp`), o time de visão (falha de classificação, `erro_classificacao`, sumidouro de M_2) e a TI (*mes-bridge* mudo, `erro_decisao`, sumidouro de M_3). Em plano **metodológico**, a disponibilização de um artefato de referência que executa o monitor sobre os cenários derivados

do contexto operacional e verifica empiricamente a Proposição 2 por *property-based testing* sobre traços gerados aleatoriamente. Como continuação, a dissertação aplicada que sucede este artigo expandirá o arcabouço para o ciclo completo da máquina, a classificação formal de paradas e os indicadores OEE/MTBF/MTTR, instanciando empiricamente o monitor composto em ambiente de produção. Em plano metodológico adicional, este trabalho introduz o padrão arquitetural *promoção de sumidouro a evento*, aplicável a extensões futuras (A6 Armed/Unarmed, A7 com efeito colateral, A8+) sem reformulação da estrutura de prova composicional.

Apêndice A: Síntese e Traços de Exemplo

A.1 Síntese via Spot

Comandos de invocação para gerar os autômatos de A1–A3:

```
ltl2tgba -f "G(rem_i -> ab_i)" # A1
ltl2tgba -f "G((leave_ab_i & houve_rem_i) \
  -> F[0..T_cls](cls_p1|cls_p2))" # A2'
ltl2tgba -f "G(leave_ab_i -> F[0..T_dec](match_i|div_i))" # A3'
# div_i = mismatch_i | fora_ciclo_i
# materializado pelo mes-bridge, fora do alfabeto Spot.
# timeout_cls_i / leave_ab_silent_i: eventos sintéticos
# promovidos dos sumidouros de M_2 / M_3 (efetor).
```

A notação $F[0..T]$ acima é abreviação: o `ltl2tgba` sintetiza apenas o *esqueto qualitativo* (com F não-temporizado); o prazo T é monitorado pelo relógio único acrescentado a cada autômato (guarda $x \leq T$, *reset* $x := 0$), conforme §5.1. Por economia notacional, os comandos omitem o índice de molde j e o limiar de A5: `rem_i` abrevia $\text{rem}_{i,j}$ e `cls_p1`, `cls_p2` abreviam $\text{cls}_{p,i,j}^{\geq \tau}$ para dois produtos representativos. O macro-evento div_i ($= \text{mismatch}_i \vee \text{fora_ciclo}_i$) e os eventos sintéticos `timeout_cls_i` e `leave_ab_silent_i` (promovidos dos sumidouros de M_2 e M_3) são materializados fora do alfabeto do Spot, pelo *mes-bridge* e pelo *efetor*.

A.2 Traços de exemplo

Aceitação. Traço σ_A com retiradas todas classificadas dentro de T_{cls} acumuladas em $M_{\text{obs}} = M_{\text{dec}}$, com *mes-bridge* emitindo match_i dentro de T_{dec} :

$$\sigma_A = \text{ab}_i \cdot \text{rem}_{i,j} \cdots \text{leave_ab}_i \cdots [\text{cls}_{p,i,j} \text{ residuais}] \cdots \text{match}_i \implies \top.$$

Violação safety (A1). $\sigma_B = \text{rem}_{i,j} \cdots$ (retirada antes de ab_i) $\Rightarrow$ sumidouro V_{A_1} .

Violação liveness temporizada (A2'). $\sigma_C = \text{ab}_i \cdot \text{rem}_{i,j} \cdot \text{leave_ab}_i \cdot \underbrace{\emptyset \cdot \emptyset \cdots}_{> T_{\text{cls}}}$ (letras vazias, sem classificação acumulada no prazo) $\Rightarrow$ sumidouro V_{A_2} , com emissão sintética de `timeout_cls_i`, que o *efetor* traduz no status `erro_classificacao` (escala à visão).

Diagnóstico em camadas (A2' vs. A3'). Sob “OP errada” sem *mes-bridge*: $\sigma_D = \text{ab}_i \cdots \text{leave_ab}_i$ (sem match_i , sem div_i , com classificação incompleta) $\Rightarrow$ sumidouro V_{A_2} (status `erro_classificacao`). O mesmo padrão de falha, em *stream* enriquecido pelo *mes-bridge* com classificação completa em T_{cls} mas sem veredito em T_{dec} , passa a satisfazer A2 e violar A3, migrando a detecção para V_{A_3} (status `erro_decisao`), o que aponta a falha à camada de integração e não ao agente de visão.

Apêndice B: Trade-off entre A2 e A2_{lat}

Duas formulações distintas para a propriedade de *liveness* de classificação podem ser consideradas: A2, com gatilho em leave_ab_i , e A2_{lat}, com gatilho em $\text{rem}_{i,j}$. A primeira pertence ao monitor composto M , pois reflete o regime de latência do mes-bridge (cf. §4); a segunda mede a *latência de inferência por retirada*, métrica útil para monitoria de drift do classificador independentemente do gate de integração MES $\leftrightarrow$ ERP. Adota-se a convenção:

- **A2:** $G((\text{leave_ab}_i \wedge \text{houve_rem}_i) \rightarrow F_{[0, T_{\text{cls}}]} \bigvee_{j,p} \text{cls}_{p,i,j}^{\geq \tau})$. Pertence ao monitor composto M . Verifica *existência agregada* de classificação válida ao final da janela acrescida do orçamento de inferência. É a propriedade que o gate consulta.
- **A2_{lat} (opcional):** $G(\text{rem}_{i,j} \rightarrow F_{[0, T_{\text{cls}}]} \bigvee_p \text{cls}_{p,i,j})$. Não pertence ao monitor composto M . Verifica *latência por retirada* para fins de observabilidade do agente. Pode ser instanciada como sub-monitor separado M_{lat} , cuja saída alimenta um painel de saúde do agente, sem participar do gate MES $\leftrightarrow$ ERP.

A separação dessas duas propriedades isola, em camadas distintas, a *decisão de produção* (A2, gate) da *telemetria do agente* (A2_{lat}, painel), reforçando o princípio arquitetural de “monitor declarativo” explorado em §5.4.

Apêndice C: Corretude composicional — demonstração completa

Esta seção desenvolve a demonstração esboçada em §5.3.

Notação. Para cada $k \in \{1, 2, 3\}$, M_k é um ω -autômato de *safety (weak) determinístico e completo* sobre $\Sigma_k = 2^{AP_k}$ ($AP_k \subseteq AP$), com um único estado-sumidouro de violação $\perp_k$ *absorvente* ($\delta_k(\perp_k, a) = \perp_k$ para todo a) e todos os demais estados aceitantes: $F_k := Q_k \setminus \{\perp_k\}$. Uma ω -palavra σ é aceita por M_k se e somente se a corrida de M_k sobre σ *nunca* atinge $\perp_k$; como $\perp_k$ é absorvente, essa condição de *safety* coincide com a condição de Büchi Inf $\subseteq F_k$ (coincidência Büchi-*weak*). Para A1, M_k tem dois estados e $\perp_1$ é alcançado por $\text{rem}_{i,j} \wedge \neg \text{ab}_i$; para A2' e A3', M_k tem três estados e $\perp_k$ é alcançado pela expiração do relógio único ($x > T$) sem o evento resolutor (*bounded liveness*, classe *safety* [Alpern and Schneider 1985, Manna and Pnueli 1992]). O macro-evento derivado $\text{div}_i \in AP$ ($= \text{mismatch}_i \vee \text{fora_ciclo}_i$) é materializado pelo mes-bridge e pertence a AP_3 (consumido por M_3). O produto sincronizado opera sobre o alfabeto comum $\Sigma = 2^{AP}$. Define-se ainda o fluxo enriquecido $\hat{\Sigma} = \Sigma \cup \{\text{timeout_cls}_i, \text{leave_ab_silent}_i\}$, consumido pelo efetor para o roteamento de status. A projeção $\pi_{\Sigma_k} : \Sigma^\omega \rightarrow \Sigma_k^\omega$ apaga as proposições fora de AP_k ; a cilindrficação $\pi_{\Sigma_k}^{-1}(L) = \{\sigma \in \Sigma^\omega : \pi_{\Sigma_k}(\sigma) \in L\}$ eleva $L \subseteq \Sigma_k^\omega$ ao alfabeto comum.

Lema 1 (a promoção de sumidouro a evento é transdução finita determinística). A aplicação $\mathcal{T} : \Sigma^\omega \rightarrow \hat{\Sigma}^\omega$ que insere os eventos sintéticos timeout_cls_i e leave_ab_silent_i nas posições determinadas pelos sumidouros de M_2 e M_3 é uma transdução de estado finito, determinística, causal e que preserva comprimento.

Prova. timeout_cls_i (resp. leave_ab_silent_i) é emitido na posição n se e somente se M_2 (resp. M_3) entra em seu único sumidouro de violação ao ler a letra $\sigma[n]$. Como M_2 e

M_3 são autômatos finitos determinísticos, o estado alcançado após $\sigma[0..n]$ é função determinística do prefixo, e a emissão depende *apenas* do estado corrente e da letra lida, sem dependência de letras futuras. Logo $\mathcal{T}$ é realizada por um transdutor de Mealy com espaço de estados finito $Q_2 \times Q_3$ — determinístico e causal. Analogamente, a materialização de div_i pelo mes-bridge, a partir do multiconjunto acumulado M_{obs} (contadores limitados, com $O(N_{\text{SKU}})$ estados) e das exceções estruturais do ciclo, é função do estado corrente e, portanto, igualmente causal. $\square$

Corolário 1. As linguagens ω -regulares são fechadas sob transdução de estado finito e sob cilindrficação; portanto $\mathcal{T}$ e $\pi_{\Sigma_k}^{-1}$ preservam ω -regularidade.

Proposição 1 (offline). Para todo $\sigma \in \Sigma^\omega$ (incluindo div_i materializado pelo mes-bridge), vale $\mathcal{L}_\omega(M) = \bigcap_{k=1}^3 \pi_{\Sigma_k}^{-1}(\mathcal{L}_\omega(M_k))$.

Prova. Por construção, M é o produto sincronizado dos três componentes elevados a Σ , e cada M_k é de *safety (weak)* determinístico com aceitação $F_k = Q_k \setminus \{\perp_k\}$. O produto é *determinístico*, logo há uma única corrida por σ , e sua componente k é exatamente a corrida de M_k sobre $\pi_{\Sigma_k}(\sigma)$ (transições sincronizadas sobre Σ). O sumidouro composto $\perp = \{(s_1, s_2, s_3) : \exists k, s_k = \perp_k\}$ é absorvente, e $F = F_1 \times F_2 \times F_3$ é o conjunto de tuplas sem coordenada em sumidouro; assim a corrida de M é aceita (evita $\perp$ para sempre) se e somente se, para todo k , a componente k nunca atinge $\perp_k$ — isto é, $\pi_{\Sigma_k}(\sigma)$ é aceita por M_k —, o que equivale a $\sigma \in \bigcap_k \pi_{\Sigma_k}^{-1}(\mathcal{L}_\omega(M_k))$. Como $\perp_k$ é absorvente, a condição “nunca atingir $\perp_k$ ” coincide com a condição de Büchi Inf $\subseteq F_k$ (coincidência Büchi–weak/co-Büchi–singleton); portanto $F = \prod_k F_k$ realiza a interseção *sem* degeneração de Büchi generalizado. O fecho por interseção das linguagens de *safety* cilindradas é o produto-padrão de autômatos de *safety* [Alpern and Schneider 1985, Baier and Katoen 2008, Havelund and Roşu 2004]; a materialização determinística de div_i pelo mes-bridge (Lema 1) garante que $\sigma \in \Sigma^\omega$ é bem-definido. $\square$

Proposição 2 (online, LTL₃). O veredito operacional do gate, computado componente-a-componente como o ínfimo na ordem $\perp < ? < \top$, $[\sigma^n \models M] := \bigcap_{k=1}^3 [\pi_{\Sigma_k}(\sigma^n) \models M_k]$, é sólido, estável e coincide com o veredito semântico LTL₃ de $\mathcal{L}_\omega(M)$.

Prova. Solidez. Se algum componente vale $\perp$, então, por [Bauer et al. 2011], nenhum prolongamento de $\pi_{\Sigma_k}(\sigma^n)$ satisfaz M_k ; pela Proposição 1, nenhum prolongamento de σ^n pertence a $\mathcal{L}_\omega(M)$, e o veredito semântico é $\perp$. Se todos os componentes valem $\top$, todo prolongamento satisfaz cada M_k e, pela Proposição 1, pertence a $\mathcal{L}_\omega(M)$; o veredito semântico é $\top$. Em ambos os casos o ínfimo coincide com o veredito semântico. **Estabilidade.** Os vereditos LTL₃ de cada M_k são estáveis [Bauer et al. 2011] — uma vez $\top$ ou $\perp$, não mudam —, e o ínfimo de vereditos estáveis é estável. **Caso ?.** O ínfimo só poderia divergir do veredito semântico se cada componente admitisse prolongamento aceitante isoladamente sem que os três *compartilhassem* um prolongamento comum — o que poderia ocorrer caso as obrigações pendentes, embora disparadas por gatilhos compartilhados (notadamente leave_ab_i , comum a $A2'$ e $A3'$), exigissem eventos resolutores mutuamente incompatíveis. O Lema 2 abaixo exclui essa hipótese, exibindo *construtiva-*

mente um único prolongamento comum para todo prefixo sem violação definitiva; logo ínfimo-? equivale a semântico-?, e a igualdade vale em todos os casos. $\square$

Lema 2 (testemunha comum). *Para todo prefixo finito $\sigma^n \in \Sigma^*$ sem violação definitiva (nenhum M_k em $\perp$), existe um sufixo $w \in \Sigma^\omega$ tal que $\sigma^n w$ satisfaz simultaneamente A1, A2' e A3'.*

Prova. Basta exibir um prolongamento aceitante (não toda extensão de σ^n): a não-vacuidade do veredito ? requer apenas a existência de algum sufixo que evite $\perp$ em cada M_k . Como nenhum M_k está em $\perp$, cada obrigação pendente em σ^n ainda admite cumprimento dentro de seu prazo — para os componentes temporizados, $M_k \not\models \perp$ equivale a relógio $x \leq T$, logo há margem para inserir o evento resolutor antes da expiração. Observe-se de início que os *gatilhos* das obrigações pendentes (*leave_ab_i*, *houve_rem_i*) já ocorreram em σ^n e não são reinseridos por w ; a ancoragem de A2' e A3' em *leave_ab_i* garante no máximo uma janela pendente por ciclo de braço (cf. §3.4), de modo que não há acúmulo de obrigações concorrentes a conciliar em paralelo. Resta, portanto, exibir eventos resolutores compatíveis. Constrói-se w na ordem temporal:

(i) Se A2' está armada e pendente (ocorreu *leave_ab_i* $\wedge$ *houve_rem_i* sem classificação válida acumulada), insira uma $\text{cls}_{p,i,j}^{\geq \tau}$ num instante $\leq \tau_b^i + T_{\text{cls}}$, resolvendo o consequente $F_{[0, T_{\text{cls}}]} \bigvee_{j,p} \text{cls}_{p,i,j}^{\geq \tau}$ de A2'. Se A2' não está armada ($\neg$ *houve_rem_i* em *leave_ab_i*, ou ainda nenhum *leave_ab_i*), a guarda do antecedente é falsa: A2' é *vacuamente satisfeita* e nenhuma $\text{cls}^{\geq \tau}$ é requerida.

(ii) Se A3' está pendente, seu consequente é a *disjunção* $F_{[0, T_{\text{dec}}]}(\text{match}_i \vee \text{div}_i)$. Diferentemente de $\text{cls}^{\geq \tau}$, os eventos *match_i* e *div_i* não são letras livremente escolhidas por w : pelo Lema 1, o mes-bridge materializa *exatamente um deles* de modo determinístico e causal, em *leave_ab_i* + ϵ (ou em $\tau_b^i + T_{\text{cls}} + \delta_{\text{mb}}$ no caso de *mismatch_i*), em função do multiconjunto observado M_{obs} contra o declarado M_{dec} e das exceções estruturais. Como A3' exige apenas a *disjunção* *match_i* $\vee$ *div_i*, *qualquer* que seja a alternativa emitida pelo mes-bridge resolve A3', contanto que seja emitida em $\leq \tau_b^i + T_{\text{dec}}$. Essa emissão é factível: o mes-bridge é a fonte determinística desses eventos e $T_{\text{dec}} \geq T_{\text{cls}}$, logo o prazo de A3' engloba o de A2' e, em particular, $\tau_b^i + T_{\text{cls}} + \delta_{\text{mb}} \leq \tau_b^i + T_{\text{dec}}$ pela hipótese $\delta_{\text{mb}} \leq T_{\text{dec}} - T_{\text{cls}}$. Concretamente, basta estender w por um sufixo no qual o mes-bridge opera normalmente: ele emitirá *match_i* na janela inerte ($M_{\text{obs}} = M_{\text{dec}} = \emptyset$, cf. §3.4) ou quando as contagens conferirem, e *div_i* caso contrário — em ambos os casos a disjunção é satisfeita.

(iii) Não insira *rem_{i,j}* fora de *ab_i*, preservando A1; e, como w não fabrica nenhuma das condições de *fora_ciclo_i* (não duplica *leave_ab_i*, não posterga *match_i* além de $\tau_b^i + T_{\text{dec}}$, não viola A1 nem a consistência M_{dec}/OP), essa exceção estrutural não dispara espuriamente.

A compatibilidade das prescrições não repousa sobre disjunção dos suportes proposicionais completos — de fato *leave_ab_i* é *gatilho compartilhado* de A2' e A3' e *houve_rem_i* é guarda de A2', de modo que os suportes se sobrepõem. A compatibilidade decorre, sim, de que (1) os gatilhos compartilhados já foram lidos em σ^n e não são reativados por w , e (2) os *eventos resolutores* efetivamente inseridos/materializados por w não colidem: A2' é resolvida por $\text{cls}^{\geq \tau}$, A3' pela alternativa *match_i*/*div_i* que o mes-bridge emite, e A1 nada exige além de w não introduzir *rem_{i,j}* $\wedge$ $\neg \text{ab}_i$. Nenhuma das

três prescrições força a falsidade de uma obrigação pendente de outra; em particular, *cls* e *match/div* podem coocorrer — sobre $\Sigma = 2^{AP}$ cada instante é um *conjunto* de proposições simultaneamente verdadeiras, não um evento atômico exclusivo —, pois $A3'$ é indiferente a qual ramo se realiza. Logo a inserção de w não reativa nenhum sumidouro já evitado e o mesmo w testemunha a satisfação simultânea dos três componentes: nenhum M_k atinge $\perp$ sobre $\sigma^n w$, e o ínfimo permanece ?. $\square$

Observação. A forma de ínfimo fornece o gate de custo $O(|\Phi|)$ por evento; a solidez garante que o gate nunca libera nem bloqueia a integração indevidamente *em relação à conjunção das especificações* (a corretude é relativa à especificação, não à fidedignidade física da inferência). O *property-based testing* relatado em §5.3 corrobora empiricamente a Proposição 2.

Declaração de uso de ferramentas de IA. Em conformidade com as recomendações de transparência editorial sobre o uso de inteligência artificial em produção acadêmica, declara-se o emprego pontual de assistentes baseados em modelos de linguagem como apoio à revisão linguística do texto e à edição técnica das figuras em TikZ. A concepção científica, a formalização das propriedades, as provas, a arquitetura proposta, os algoritmos e a análise crítica dos resultados são de autoria e responsabilidade integral do autor.

Referências

- Alpern, B. and Schneider, F. B. (1985). Defining liveness. *Information Processing Letters*, 21(4):181–185. DOI: [https://doi.org/10.1016/0020-0190\(85\)90056-0](https://doi.org/10.1016/0020-0190(85)90056-0)
- Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., and Topcu, U. (2018). Safe reinforcement learning via shielding. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 2669–2678. AAAI Press. DOI: <https://doi.org/10.1609/aaai.v32i1.11797>
- Alur, R. and Dill, D. L. (1994). A theory of timed automata. *Theoretical Computer Science*, 126(2):183–235. DOI: [https://doi.org/10.1016/0304-3975\(94\)90010-8](https://doi.org/10.1016/0304-3975(94)90010-8)
- Alur, R., Feder, T., and Henzinger, T. A. (1996). The benefits of relaxing punctuality. *Journal of the ACM*, 43(1):116–146. DOI: <https://doi.org/10.1145/227595.227602>
- Alur, R. and Henzinger, T. A. (1994). A really temporal logic. *Journal of the ACM*, 41(1):181–204. DOI: <https://doi.org/10.1145/174644.174651>
- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. (2016). Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*.
- Baier, C. and Katoen, J.-P. (2008). *Principles of Model Checking*. MIT Press, Cambridge, MA.
- Bartocci, E., Falcone, Y., Francalanza, A., and Reger, G. (2018). Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*,

volume 10457 of LNCS, pages 1–33. Springer. DOI: https://doi.org/10.1007/978-3-319-75632-5_1

- Bauer, A., Leucker, M., and Schallhart, C. (2007). The good, the bad, and the ugly, but how ugly is ugly? In *Proceedings of the 7th International Workshop on Runtime Verification (RV)*, volume 4839 of LNCS, pages 126–138. Springer. DOI: https://doi.org/10.1007/978-3-540-77395-5_11
- Bauer, A., Leucker, M., and Schallhart, C. (2011). Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology*, 20(4):14:1–14:64. DOI: <https://doi.org/10.1145/2000799.2000800>
- Büchi, J. R. (1962). On a decision method in restricted second-order arithmetic. In *Proceedings of the International Congress on Logic, Methodology and Philosophy of Science*, pages 1–11. Stanford University Press.
- Cardin, O. (2019). Classification of cyber-physical production systems applications: proposition of an analysis framework. *Computers in Industry*, 104:11–21. DOI: <https://doi.org/10.1016/j.compind.2018.10.002>
- Cheng, C.-H., Nührenberg, G., and Yasuoka, H. (2019). Runtime monitoring neuron activation patterns. In *Proceedings of the Design, Automation & Test in Europe Conference (DATE)*. IEEE. DOI: <https://doi.org/10.23919/DATE.2019.8714971>
- Choueka, Y. (1974). Theories of automata on ω -tapes: a simplified approach. *Journal of Computer and System Sciences*, 8(2):117–141. DOI: [https://doi.org/10.1016/S0022-0000\(74\)80051-6](https://doi.org/10.1016/S0022-0000(74)80051-6)
- Cimatti, A., Clarke, E., Giunchiglia, E., Giunchiglia, F., Pistore, M., Roveri, M., Sebastiani, R., and Tacchella, A. (2002). NuSMV 2: an opensource tool for symbolic model checking. In *Proceedings of the 14th International Conference on Computer Aided Verification (CAV)*, volume 2404 of LNCS, pages 359–364. Springer. DOI: https://doi.org/10.1007/3-540-45657-0_29
- Convent, L., Hungerecker, S., Leucker, M., Scheffel, T., Schmitz, M., and Thoma, D. (2018). TeSSLa: temporal stream-based specification language. In *Formal Methods: Foundations and Applications (SBMF)*, volume 11254 of LNCS, pages 144–162. Springer. DOI: https://doi.org/10.1007/978-3-030-03044-5_10
- D’Angelo, B., Sankaranarayanan, S., Sánchez, C., Robinson, W., Finkbeiner, B., Sipma, H. B., Mehrotra, S., and Manna, Z. (2005). LOLA: runtime monitoring of synchronous systems. In *Proceedings of the 12th International Symposium on Temporal Representation and Reasoning (TIME)*, pages 166–174. IEEE. DOI: <https://doi.org/10.1109/TIME.2005.26>
- Duret-Lutz, A., Lewkowicz, A., Fauchille, A., Michaud, T., Renault, E., and Xu, L. (2016). Spot 2.0 — a framework for LTL and ω -automata manipulation. In *Proceedings of the 14th International Symposium on Automated Technology for Verification and Analysis (ATVA)*, volume 9938 of LNCS, pages 122–129. Springer. DOI: https://doi.org/10.1007/978-3-319-46520-3_8

- Falcone, Y., Fernandez, J.-C., and Mounier, L. (2012). What can you verify and enforce at runtime? *International Journal on Software Tools for Technology Transfer (STTT)*, 14(3):349–382. DOI: <https://doi.org/10.1007/s10009-011-0196-8>
- Havelund, K. and Roşu, G. (2004). Efficient monitoring of safety properties. *International Journal on Software Tools for Technology Transfer*, 6(2):158–173. DOI: <https://doi.org/10.1007/s10009-003-0117-6>
- Henzinger, T. A., Lukina, A., and Schilling, C. (2020). Outside the box: Abstraction-based monitoring of neural networks. In *Proceedings of the 24th European Conference on Artificial Intelligence (ECAI)*, volume 325 of Frontiers in Artificial Intelligence and Applications. IOS Press. DOI: <https://doi.org/10.3233/FAIA200375>
- Holzmann, G. J. (2003). *The Spin Model Checker: Primer and Reference Manual*. Addison-Wesley, Boston, MA.
- Jeon, B. W., Um, J., Yoon, S. C., and Suh, S.-H. (2017). An architecture design for smart manufacturing execution system. *Computer-Aided Design and Applications*, 14(4):472–485. DOI: <https://doi.org/10.1080/16864360.2016.1257189>
- Junges, S., Jansen, N., and Seshia, S. A. (2021). Enforcing almost-sure reachability in POMDPs. In *Proceedings of the 33rd International Conference on Computer Aided Verification (CAV)*, volume 12760 of LNCS, pages 602–625. Springer. DOI: https://doi.org/10.1007/978-3-030-81688-9_28
- Koymans, R. (1990). Specifying real-time properties with metric temporal logic. *Real-Time Systems*, 2(4):255–299. DOI: <https://doi.org/10.1007/BF01995674>
- Lamport, L. (1977). Proving the correctness of multiprocess programs. *IEEE Transactions on Software Engineering*, SE-3(2):125–143. DOI: <https://doi.org/10.1109/TSE.1977.229904>
- Leitão, P. (2009). Agent-based distributed manufacturing control: a state-of-the-art survey. *Engineering Applications of Artificial Intelligence*, 22(7):979–991. DOI: <https://doi.org/10.1016/j.engappai.2008.09.005>
- Leitão, P., Karnouskos, S., Ribeiro, L., Lee, J., Strasser, T., and Colombo, A. W. (2016). Smart agents in industrial cyber-physical systems. *Proceedings of the IEEE*, 104(5):1086–1101. DOI: <https://doi.org/10.1109/JPROC.2016.2521931>
- Leucker, M. and Schallhart, C. (2009). A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303. DOI: <https://doi.org/10.1016/j.jlap.2008.08.004>
- Lukina, A., Schilling, C., and Henzinger, T. A. (2021). Into the unknown: Active monitoring of neural networks. In *Proceedings of the 21st International Conference on Runtime Verification (RV)*, volume 12974 of LNCS. Springer. DOI: https://doi.org/10.1007/978-3-030-88494-9_3

- Maler, O. and Nickovic, D. (2004). Monitoring temporal properties of continuous signals. In *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT)*, volume 3253 of LNCS, pages 152–166. Springer. DOI: https://doi.org/10.1007/978-3-540-30206-3_12
- Manna, Z. and Pnueli, A. (1992). *The Temporal Logic of Reactive and Concurrent Systems: Specification*. Springer-Verlag. DOI: <https://doi.org/10.1007/978-1-4612-0931-7>
- Monostori, L., Váncza, J., and Kumara, S. R. T. (2006). Agent-based systems for manufacturing. *CIRP Annals*, 55(2):697–720. DOI: <https://doi.org/10.1016/j.cirp.2006.10.004>
- Pinisetty, S., Roop, P. S., Smyth, S., Allen, N., Tripakis, S., and von Hanxleden, R. (2017). Runtime enforcement of cyber-physical systems. *ACM Transactions on Embedded Computing Systems*, 16(5s):1–25. DOI: <https://doi.org/10.1145/3126500>
- Pnueli, A. (1977). The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 46–57. IEEE. DOI: <https://doi.org/10.1109/SFCS.1977.32>
- Pnueli, A. and Zaks, A. (2006). PSL model checking and run-time verification via testers. In *Proceedings of the 14th International Conference on Formal Methods (FM)*, volume 4085 of LNCS, pages 573–586. Springer. DOI: https://doi.org/10.1007/11813040_38
- Schneider, F. B. (2000). Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50. DOI: <https://doi.org/10.1145/353323.353382>
- Törngren, M., Thompson, H., Herzog, E., Inam, R., Gross, J., and Dán, G. (2021). Industrial edge-based cyber-physical systems: application needs and concerns for realization. In *Proceedings of the 2021 IEEE/ACM Symposium on Edge Computing (SEC)*, pages 409–415. IEEE.
- Van Brussel, H., Wyns, J., Valckenaers, P., Bongaerts, L., and Peeters, P. (1998). Reference architecture for holonic manufacturing systems: PROSA. *Computers in Industry*, 37(3):255–274. DOI: [https://doi.org/10.1016/S0166-3615\(98\)00102-X](https://doi.org/10.1016/S0166-3615(98)00102-X)
- Vardi, M. Y. and Wolper, P. (1986). An automata-theoretic approach to automatic program verification. In *Proceedings of the 1st IEEE Symposium on Logic in Computer Science (LICS)*, pages 332–344. IEEE.
- Younes, H. L. S. and Simmons, R. G. (2002). Probabilistic verification of discrete event systems using acceptance sampling. In *Proceedings of the 14th International Conference on Computer Aided Verification (CAV)*, volume 2404 of LNCS, pages 223–235. Springer. DOI: https://doi.org/10.1007/3-540-45657-0_17
- Angelopoulos, A. N. and Bates, S. (2023). Conformal prediction: a gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591. DOI: <https://doi.org/10.1561/22000000101>

- Gibbs, I. and Candès, E. (2021). Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 1660–1672.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330.
- Vovk, V., Gammerman, A., and Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer, New York. DOI: <https://doi.org/10.1007/b106715>